

**Diseño e Implementación de un Agente en Espacio de
Usuario**
**para la Gestión Dinámica de Frecuencia (DVFS) en
Sistemas**
**Heterogéneos mediante Modelos Ligeros de Machine
Learning**

Yeison Adrián Cáceres Torres

Ricardo Andrés Pérez Porras

Trabajo de Grado para Optar el Título de Ingeniero de Sistemas

Director

Gilberto Javier Díaz Toro

Doctor en Ingeniería

Universidad Industrial de Santander

Facultad de Ingenierías Físico-Mecánicas

Escuela de Ingeniería de Sistemas e Informática

Bucaramanga

2026

Agradecimientos

Resumen

Título: Diseño e Implementación de un Agente en Espacio de Usuario para la Gestión Dinámica de Frecuencia (DVFS) en Sistemas Heterogéneos mediante Modelos Ligeros de Machine Learning.

Autores: Yeison Adrián Cáceres Torres; Ricardo Andrés Pérez Porras.

Palabras Clave: DVFS; eficiencia energética; computación de alto rendimiento; modelo Roofline; telemetría microarquitectónica; aprendizaje automático supervisado.

Descripción:

El consumo energético constituye hoy una restricción estructural para el escalamiento de la computación de alto rendimiento (HPC), particularmente en nodos heterogéneos que integran CPU y aceleradores gráficos. El Escalado Dinámico de Voltaje y Frecuencia (DVFS) es el principal mecanismo de control disponible a nivel de hardware, pero su aprovechamiento efectivo depende de decidir, durante la ejecución, cuál punto operativo conviene según el comportamiento de la carga. Los gobernadores nativos del sistema operativo deciden a partir de la utilización agregada del procesador, una señal que no distingue si el rendimiento está limitado por la capacidad aritmética del núcleo o por la transferencia de datos desde memoria; en el segundo caso, sostener la frecuencia máxima incrementa el consumo sin una mejora proporcional del tiempo de ejecución.

Este trabajo propone el diseño, la implementación y la evaluación de un agente en espacio de usuario que infiere el régimen de ejecución de la aplicación a partir de telemetría microarquitectónica y traduce esa inferencia en decisiones discretas de DVFS sobre CPU y GPU, con el objetivo de optimizar el Producto Energía–Retardo (EDP) sin incurrir en una penalización severa del rendimiento ni en una sobrecarga de inferencia que anule el beneficio

energético.

El desarrollo se organiza en cuatro fases: caracterización de cargas y construcción del conjunto de datos; entrenamiento y selección de un clasificador supervisado ligero; implementación del servicio de control; y validación experimental frente a los gobernadores nativos de Linux. El presente documento reporta el instrumento de medición y el protocolo experimental contruidos durante la primera fase — cuyo aporte metodológico central es un procedimiento reproducible para etiquetar como *compute-bound* o *memory-bound* cada ventana de ejecución de CPU o corrida completa de GPU, mediante una calibración empírica del modelo Roofline sobre la plataforma de medición, con trazabilidad explícita de la calidad de cada observación — junto con un primer entrenamiento y análisis del clasificador de GPU, posible ya dentro de esta fase por disponer de su conjunto de entrenamiento definido.

Abstract

Title: Design and Implementation of a User-Space Agent for Dynamic Frequency Management (DVFS) in Heterogeneous Systems through Lightweight Machine Learning Models.

Authors: Yeison Adrián Cáceres Torres; Ricardo Andrés Pérez Porras.

Keywords: DVFS; energy efficiency; high performance computing; Roofline model; micro-architectural telemetry; supervised machine learning.

Description:

Energy consumption has become a structural constraint on the scaling of high performance computing (HPC), particularly on heterogeneous nodes that combine CPUs and graphics accelerators. Dynamic Voltage and Frequency Scaling (DVFS) is the main control mechanism available at the hardware level, but exploiting it effectively requires deciding, at run time, which operating point suits the current workload behaviour. Native operating system governors base that decision on aggregate processor utilisation, a signal that does not distinguish whether performance is limited by the arithmetic capability of the core or by data transfer from memory; in the latter case, sustaining maximum frequency raises power draw without a proportional improvement in execution time.

This work proposes the design, implementation and evaluation of a user-space agent that infers the execution regime of an application from microarchitectural telemetry and translates that inference into discrete DVFS decisions over CPU and GPU, aiming to optimise the Energy-Delay Product (EDP) without incurring a severe performance penalty or an inference overhead that cancels the energy benefit.

The work is organised in four phases: workload characterisation and dataset construction; training and selection of a lightweight supervised classifier; implementation of the control

service; and experimental validation against the native Linux governors. This document reports the measurement instrument and experimental protocol built during the first phase, whose central methodological contribution is a reproducible procedure for labelling execution windows as *compute-bound* or *memory-bound* through an empirical calibration of the Roofline model on the measurement platform, with explicit traceability of the quality of each observation.

Índice general

Agradecimientos	1
Resumen	2
Abstract	4
Introducción	10
Planteamiento y justificación del problema	11
Objetivos	12
1. Marco de referencia	14
1.1. Marco Conceptual	14
1.1.1. Potencia CMOS y fundamento de DVFS	14
1.1.2. Modelo Roofline y regímenes de ejecución	15
1.1.3. Telemetría microarquitectónica e instrumentación	16
1.1.4. Control DVFS desde espacio de usuario	18
1.1.5. Cargas de trabajo y calibración	19
1.1.6. Clasificación supervisada ligera	19
1.1.7. Optimización de hiperparámetros y Optuna	20
1.1.8. Producto Energía–Retardo	21
1.2. Estado del Arte	22
2. Metodología	25
2.1. Enfoque metodológico	25
2.2. Diseño metodológico	25
2.2.1. Población y muestra	25
2.2.2. Plataforma experimental	26
2.2.3. Instrumentos de recolección	27

2.2.4.	Variables observadas	30
2.3.	Fase 1: Recolección de información y caracterización de cargas	33
2.3.1.	Verificación previa de la plataforma	33
2.3.2.	Catálogo de cargas de trabajo	33
2.3.3.	Selección preliminar y definición del catálogo definitivo	35
2.3.4.	Calibración y criterio de etiquetado	36
2.3.5.	Caracterización de intensidad operacional y calibración Roofline en GPU	36
2.3.6.	Dimensionamiento de las cargas GPU y verificación de invarianza de la intensidad operacional	37
2.3.7.	Observabilidad efectiva y latencia de conmutación de reloj en GPU .	38
2.3.8.	Matriz experimental y control de sesgos	38
2.3.9.	Post-procesamiento y control de calidad de los datos	40
2.3.10.	Unidad de observación del conjunto de entrenamiento	41
2.3.11.	Medición directa de FLOPs por ventana y su validación empírica . .	42
2.3.12.	Reproducibilidad	43
2.3.13.	Selección y registro de características	43
2.4.	Fase 2: Desarrollo del modelo de aprendizaje automático	44
2.5.	Fase 3: Implementación del agente de control DVFS	45
2.5.1.	Arquitectura de dos bucles desacoplados	45
2.5.2.	Detección de fase en GPU	45
2.5.3.	Diseño del bucle de CPU e integración con el clasificador	45
2.5.4.	Derivación <i>offline</i> de la tabla de política	45
2.5.5.	Actuación y validación en modo simulado	46
2.6.	Fase 4: Validación experimental y análisis de resultados	46
2.6.1.	Escenarios de comparación	46
2.6.2.	Conmutación de gobernadores y protocolo de medición	46
2.6.3.	Cálculo del EDP y prueba de significancia	46
2.6.4.	Generación del reporte comparativo	47
2.7.	Aspectos éticos	47
3.	Conclusiones	48
3.1.	Limitaciones	49
3.2.	Trabajo Futuro	49
	Repositorio y recursos complementarios	56

Índice de tablas

2.1. Características de la plataforma experimental.	27
2.2. Señales de telemetría de CPU activas en el instrumento de producción. . . .	32
2.3. Catálogo de cargas de trabajo.	34
2.4. Rejilla fina de frecuencia de la campaña definitiva.	38
2.5. Taxonomía de clasificación de frecuencia por ventana de muestreo.	39
2.6. Taxonomía de anotaciones de calidad por ventana de muestreo.	41
2.7. FLOPs por instrucción retirada según ancho vectorial y tipo de operación, evento FP_ARITH_INST_RETIRED de Ice Lake-SP. La columna FMA ya está incorporada en el valor que reporta el contador de hardware.	43

Índice de figuras

1.1. Modelo Roofline conceptual	15
2.1. Ruta de muestreo con estructura circular de un productor y un consumidor .	29
2.2. Recorrido de una campaña a través de las dos capas del instrumento	30

Introducción

En los sistemas de cómputo de alto rendimiento (HPC), el consumo energético se ha consolidado como una restricción crítica para el escalado sostenible, especialmente en arquitecturas heterogéneas CPU–GPU. En este contexto, el Escalado Dinámico de Voltaje y Frecuencia (DVFS) constituye el principal mecanismo de control a nivel de hardware, permitiendo ajustar los estados operativos de los procesadores para optimizar el compromiso entre consumo energético y tiempo de ejecución. Sin embargo, la efectividad de este mecanismo depende de la capacidad del sistema para adaptar dinámicamente la frecuencia al comportamiento cambiante de las aplicaciones, lo cual representa un desafío abierto en entornos HPC modernos.

En la literatura, este problema ha sido abordado principalmente mediante dos enfoques. Por un lado, los gobernadores de energía del sistema operativo, como *ondemand* o *schedutil*, emplean políticas reactivas basadas principalmente en la utilización del procesador, sin considerar la naturaleza microarquitectónica de la carga de trabajo, lo que limita su capacidad de adaptación a cargas de trabajo científicas dinámicas. Por otro lado, diversas propuestas han explorado estrategias heurísticas basadas en métricas microarquitectónicas, utilizando reglas deterministas para ajustar la frecuencia de operación. Aunque estos métodos presentan bajo *overhead*, su dependencia de umbrales estáticos y su limitada capacidad de generalización reducen su efectividad en escenarios donde las fases computacionales varían rápidamente.

A pesar de estos avances, persiste una limitación fundamental: la incapacidad de los enfoques actuales para capturar de forma robusta las transiciones dinámicas entre regímenes *compute-bound* y *memory-bound*, así como para adaptarse a la heterogeneidad CPU–GPU sin intervención manual o ajuste específico por carga de trabajo. Esta brecha evidencia la necesidad de mecanismos de control más flexibles que permitan inferir el régimen computacional de la aplicación en tiempo de ejecución y tomar decisiones de escalado de frecuencia fundamentadas en el perfil de ejecución.

En este contexto, el presente trabajo propone el diseño e implementación de un agente en

espacio de usuario basado en modelos ligeros de aprendizaje automático, capaz de clasificar en tiempo de ejecución las fases de una aplicación como compute-bound o memory-bound a partir de telemetría microarquitectónica, y de ajustar dinámicamente la frecuencia de CPU y GPU para optimizar el Producto Energía–Retardo (EDP). Se plantea demostrar que este enfoque mejora la eficiencia energética frente a los gobernadores nativos de Linux, manteniendo una degradación controlada del rendimiento y una sobrecarga de inferencia mínima y cuantificada mediante métricas experimentales.

Planteamiento y justificación del problema

Existe una ineficiencia energética estructural en la operación de los sistemas de alto rendimiento (HPC), especialmente en infraestructuras que integran aceleradores gráficos (GPU) para ejecutar aplicaciones científicas intensivas. Cuando una aplicación entra en una fase donde el procesamiento depende principalmente de la transferencia de datos desde memoria, mantener la CPU o la GPU a su frecuencia máxima no mejora el rendimiento de manera proporcional. En estas condiciones, los núcleos de cómputo permanecen parcialmente inactivos esperando datos, lo que genera un consumo energético elevado sin un beneficio equivalente en tiempo de ejecución. Esta desalineación entre demanda computacional y frecuencia operativa ha motivado el estudio del escalado de frecuencia como mecanismo para mejorar la eficiencia energética en aplicaciones científicas [39] y en sistemas de gran escala donde la gestión de potencia impacta el comportamiento global del sistema [13].

El problema se agrava por la naturaleza multifásica de las aplicaciones HPC, cuyo perfil de ejecución cambia a lo largo del tiempo de ejecución. En particular, estas aplicaciones alternan entre fases donde el rendimiento está limitado por la capacidad de cómputo del procesador (*compute-bound*) y fases donde está restringido por la transferencia de datos desde memoria (*memory-bound*). Esta distinción es crítica porque, en el régimen *memory-bound*, incrementos de frecuencia tienden a producir mejoras marginales en el tiempo de ejecución, mientras aumentan el consumo energético; en contraste, en fases *compute-bound* la frecuencia sí impacta el desempeño de forma más directa. Por ello, se han propuesto marcos de caracterización para identificar y clasificar estas fases en entornos HPC, con el fin de habilitar decisiones de control más informadas [3, 9].

A partir de este contexto, la gestión eficiente de frecuencia no depende únicamente de la disponibilidad del mecanismo DVFS en el hardware, sino de la capacidad del sistema para decidir, en tiempo de ejecución, cuál configuración resulta más conveniente según el comportamiento de la carga. Aunque se han propuesto mecanismos automatizados para seleccionar

frecuencias óptimas de GPU con el fin de mejorar la eficiencia energética sin comprometer significativamente el desempeño [2], persiste una brecha en la implementación de estrategias que integren de forma coordinada componentes CPU–GPU y que operen durante la ejecución de la aplicación, considerando además el costo computacional asociado al propio proceso de decisión.

Esta brecha resulta especialmente relevante en entornos académicos y científicos, donde el consumo eléctrico incide directamente en los costos operativos, la disponibilidad efectiva de recursos y la sostenibilidad institucional. En este contexto, contar con un mecanismo capaz de observar el comportamiento de la aplicación, inferir su fase de ejecución y ajustar la frecuencia de operación sin intervenir el código fuente constituye una alternativa con valor técnico y práctico.

Desde la perspectiva internacional, la literatura reciente ha mostrado avances en la optimización energética mediante escalado de frecuencia, gestión de potencia y caracterización de cargas HPC [39, 13, 3, 9, 2]. Sin embargo, a nivel nacional y regional, este tipo de soluciones aún no se encuentra ampliamente implementado en infraestructuras académicas de cómputo científico, particularmente bajo esquemas que combinen telemetría de bajo nivel, modelos ligeros de aprendizaje automático y control en espacio de usuario. En consecuencia, el problema no solo es técnicamente relevante, sino también pertinente en términos de aplicabilidad local.

A partir de esta necesidad de optimización energética en sistemas heterogéneos, se formula la siguiente pregunta de investigación que servirá como eje de desarrollo del proyecto:

¿En qué medida el diseño e implementación de un agente en espacio de usuario, guiado por modelos ligeros de aprendizaje automático, permite ajustar la frecuencia de operación (DVFS) en sistemas heterogéneos CPU–GPU para reducir el consumo energético, sin que el costo de la inferencia computacional degrade el rendimiento global de la aplicación, en comparación con los gobernadores nativos de Linux?

Objetivos

Objetivo General

Caracterizar, diseñar, implementar y evaluar un agente en espacio de usuario basado en modelos ligeros de Aprendizaje Automático para el ajuste dinámico de frecuencia (DVFS)

en sistemas heterogéneos CPU–GPU, con el propósito de maximizar la eficiencia energética en aplicaciones científicas sin inducir una penalización severa en su rendimiento global.

Objetivos Específicos

1. Caracterizar el comportamiento computacional y el consumo energético de cargas de trabajo representativas (intensivas en cómputo y en memoria) bajo distintos estados de frecuencia, recolectando telemetría de bajo nivel mediante contadores de rendimiento por hardware e interfaces de potencia estándar (Perf y RAPL para CPU y NVML para GPU).
2. Entrenar y validar modelos clásicos de Aprendizaje Automático (tales como Árboles de Decisión o Bosques Aleatorios) que sean capaces de clasificar, en tiempo de ejecución y con baja latencia de inferencia, las fases de ejecución de las aplicaciones basándose en la telemetría extraída.
3. Desarrollar un servicio de control (*daemon*) en el espacio de usuario que interactúe de forma asíncrona con el sistema operativo, leyendo el estado de los contadores de hardware, ejecutando la inferencia del modelo clasificador y aplicando políticas proactivas de DVFS a través de las interfaces estándar del sistema operativo, en función de la fase de ejecución inferida por el modelo.
4. Evaluar el impacto empírico del sistema propuesto mediante el cálculo del Producto Energía–Retardo (EDP), con el fin de determinar estadísticamente si el ahorro energético compensa la sobrecarga computacional (*overhead*) derivado de la inferencia del modelo en comparación con los gobernadores nativos de Linux.

Capítulo 1

Marco de referencia

1.1. Marco Conceptual

El ajuste dinámico de frecuencia y voltaje parte de la relación entre el punto de operación del hardware, la potencia consumida y el tiempo de ejecución. El efecto de modificar ese punto depende del comportamiento de la carga, por lo que resulta necesario establecer qué limita su desempeño y qué señales permiten reconocer esa condición durante la ejecución. A partir de esta relación se organiza el marco conceptual, comenzando por el modelo Roofline y la caracterización de los regímenes *compute-bound* y *memory-bound*, para continuar con la telemetría disponible en CPU y GPU, los modelos ligeros utilizados para inferir ese comportamiento y los mecanismos de control DVFS. Finalmente, se introduce el Producto Energía–Retardo como referencia para relacionar el ahorro energético con el efecto que una decisión de frecuencia produce sobre el tiempo de ejecución.

1.1.1. Potencia CMOS y fundamento de DVFS

La potencia consumida por un circuito CMOS incluye una componente estática, asociada principalmente a corrientes de fuga, y una componente dinámica producida por la conmutación de cargas capacitivas. Dado que DVFS modifica la frecuencia y, según el estado operativo, también el voltaje de alimentación, su efecto se relaciona de forma directa con la potencia dinámica, que puede aproximarse como:

$$P_{\text{dyn}} = \alpha \cdot C \cdot V^2 \cdot f \quad (1.1)$$

donde α representa el factor de actividad, C la capacitancia efectiva conmutada, V

el voltaje de alimentación y f la frecuencia de operación [17]. La dependencia cuadrática respecto al voltaje explica por qué una reducción del punto operativo puede disminuir de forma apreciable la potencia, aunque ese beneficio no puede evaluarse sin considerar el tiempo de ejecución. Una carga que pierde rendimiento al reducir la frecuencia puede permanecer activa durante más tiempo y compensar parte del ahorro obtenido. Esta relación física da origen al compromiso energía–rendimiento que debe resolver una política DVFS.

1.1.2. Modelo Roofline y regímenes de ejecución

El efecto de variar la frecuencia no es uniforme porque el rendimiento de una carga puede estar condicionado por recursos distintos. El modelo Roofline formaliza esta diferencia al relacionar la intensidad operacional con los límites de cómputo y ancho de banda de memoria de la plataforma [46]. Su formulación básica es:

$$P = \min(P_{\text{pico}}, I \cdot BW_{\text{pico}}) \quad (1.2)$$

donde P_{pico} corresponde al rendimiento aritmético pico, I a la intensidad operacional y BW_{pico} al ancho de banda pico de memoria. Cuando el límite viene dado por $I \cdot BW_{\text{pico}}$, aumentar la capacidad de cómputo no elimina la restricción impuesta por el movimiento de datos y la carga se considera *memory-bound*. Cuando el límite corresponde a P_{pico} , el rendimiento queda ligado a la capacidad aritmética y el régimen se considera *compute-bound*.

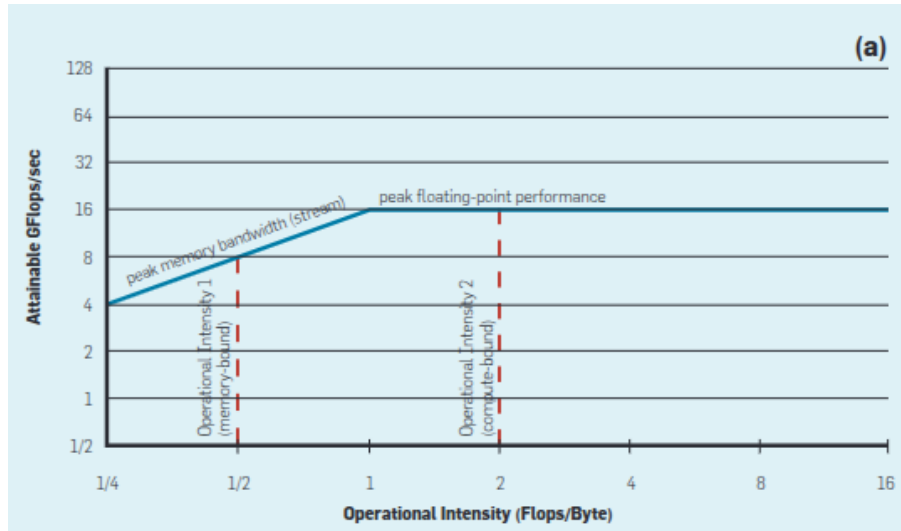


Figura 1.1: Modelo Roofline conceptual y relación entre el techo de ancho de banda de memoria y el techo de rendimiento de cómputo. Tomado de Williams et al. [46, Fig. 1(a)].

La separación entre ambos regímenes se encuentra en el *ridge point*, obtenido al igualar

las dos ramas del modelo:

$$I_{\text{ridge}} = \frac{P_{\text{pico}}}{BW_{\text{pico}}} \quad (1.3)$$

Una intensidad inferior a I_{ridge} sitúa la carga en la región limitada por memoria y una intensidad superior en la región limitada por cómputo [46]. Este criterio es especialmente útil para el proyecto porque permite construir una etiqueta a partir de una referencia física del hardware, en lugar de asumir el régimen por el nombre del kernel o por su comportamiento esperado. El punto de inflexión tampoco debe tratarse como una constante universal. El techo de cómputo depende de la precisión aritmética y puede variar con la frecuencia, mientras que el ancho de banda pertenece a otro dominio y no necesariamente cambia en la misma proporción [20, 2]. Por ello, la interpretación de una carga debe mantenerse vinculada a los techos correspondientes a la plataforma y al punto operativo evaluado.

La intensidad operacional observada durante un intervalo i se calcula como:

$$I_i = \frac{\text{FLOPs}_i}{B_i^{\text{DRAM}}} \quad (1.4)$$

FLOPs_i representa el trabajo aritmético realizado y B_i^{DRAM} los bytes transferidos hacia o desde memoria principal durante el mismo intervalo [46]. Esta correspondencia temporal es necesaria para conservar el significado físico de la razón. En CPU, el trabajo aritmético puede obtenerse mediante eventos de rendimiento y el tráfico hacia DRAM mediante contadores del controlador de memoria. En GPU, la obtención de ambas magnitudes requiere una instrumentación de mayor detalle que la telemetría ligera disponible durante la ejecución. Esta diferencia explica por qué la referencia Roofline y las señales usadas posteriormente para inferir el régimen no proceden necesariamente de la misma fuente.

1.1.3. Telemetría microarquitectónica e instrumentación

La clasificación en línea requiere señales que reflejen el comportamiento del hardware sin reproducir directamente el cálculo de la etiqueta Roofline. En CPU, estas señales se obtienen principalmente mediante las Unidades de Monitorización de Rendimiento, o PMU, que registran eventos asociados a la ejecución. Parte de los contadores se reserva para eventos frecuentes, como ciclos e instrucciones retiradas, mientras que otros son programables y permiten seleccionar eventos definidos por la microarquitectura [24]. Esta organización ofrece flexibilidad, pero también impone un límite al número de eventos que pueden observarse al

mismo tiempo.

Cuando la cantidad de eventos solicitados supera los contadores físicos disponibles, Linux puede multiplexarlos y estimar los conteos a partir del tiempo durante el cual cada evento estuvo habilitado y realmente activo [32]. La relación entre *time running* y *time enabled* permite reconocer esa situación y valorar si la lectura mantiene una cobertura suficiente [32, 31]. El problema no es únicamente que existan pocos contadores, sino que una señal observada de forma intermitente puede representar mal una carga cuyo comportamiento cambia durante los periodos no medidos. Esta limitación condiciona qué eventos resultan adecuados para alimentar un clasificador que pretende operar sobre ventanas de ejecución.

La información tampoco procede de un único dominio. Los contadores de núcleo registran eventos ligados a la actividad de los procesadores lógicos y permiten derivar señales como IPC, tasa de fallos de caché, MPKI o ciclos de espera. Los eventos *uncore*, en cambio, cubren recursos compartidos del zócalo, entre ellos el controlador de memoria y las interconexiones [22]. Esta diferencia permite observar el tráfico que alcanza DRAM, pero hace que su alcance y granularidad no coincidan necesariamente con los contadores de núcleo. Para calcular intensidad operacional, el trabajo aritmético y los bytes de memoria deben agregarse sobre límites temporales compatibles antes de aplicar el criterio Roofline.

Las métricas derivadas de los eventos de núcleo cumplen una función distinta a la etiqueta. Un IPC bajo puede estar asociado con esperas, dependencias o presión de memoria, mientras que los fallos de caché y los ciclos con solicitudes de memoria pendientes aportan señales adicionales sobre la forma en que progresa la ejecución [19, 23]. Ninguna de estas variables demuestra por sí sola que una carga sea *memory-bound*. En Hyperion se utilizan como observaciones a partir de las cuales el clasificador intenta aproximar una clase definida de manera independiente mediante Roofline. Mantener esta separación evita convertir un indicador microarquitectónico en una verdad física que no representa por sí mismo.

Linux expone estas mediciones mediante `perf_event`, que permite acceder desde espacio de usuario a eventos de la PMU sin manipular directamente sus registros [31, 45]. Para la energía de CPU, Intel RAPL proporciona contadores acumulativos asociados a dominios disponibles en la plataforma, entre ellos *Package* y, cuando existe soporte, DRAM [14, 41]. La diferencia entre dos lecturas permite obtener la energía correspondiente a un intervalo, aunque esta medición representa los dominios expuestos por RAPL y no el consumo eléctrico completo del nodo.

La observabilidad de GPU tiene otra granularidad. NVML ofrece desde espacio de usuario métricas agregadas de utilización, actividad de memoria, potencia, energía y frecuencias

de operación [36]. Son adecuadas para seguimiento continuo porque no requieren instrumentar cada kernel, pero no proporcionan directamente los FLOPs y bytes necesarios para reconstruir la intensidad operacional. Nsight Compute, por el contrario, puede perfilar kernels CUDA con métricas de mayor detalle y obtener información suficiente para caracterizar su posición en Roofline [37]. Su costo de perfilado, que puede incluir varias pasadas sobre un mismo kernel, hace que su papel sea distinto. `ncu` proporciona la referencia fuera de línea y NVML las señales disponibles durante la inferencia, una separación que mantiene la verdad de entrenamiento independiente de los proxies utilizados por el modelo.

1.1.4. Control DVFS desde espacio de usuario

El agente opera en espacio de usuario, mientras que el acceso directo a los mecanismos del hardware permanece bajo control del kernel y de los controladores del dispositivo [27, 35]. Esta separación evita incorporar la lógica del proyecto como un módulo del kernel y permite trabajar sobre interfaces estándar del sistema, aunque cada lectura, inferencia y solicitud de frecuencia añade tiempo al lazo de decisión. La utilidad del control depende, por tanto, de que ese costo sea pequeño frente a la duración del comportamiento que se intenta aprovechar.

En CPU, Linux organiza la gestión de frecuencia mediante `CPUFreq` y los mecanismos soportados por el procesador. Los *governors* determinan cómo se solicita un nivel de rendimiento, mientras que el controlador traduce esa petición al hardware [21, 29]. El *governor userspace*, cuando está disponible, permite que una política externa participe en esa decisión, y las interfaces de `CPUFreq` también exponen límites sobre los que puede actuar un servicio en espacio de usuario. Hyperion se apoya en estas interfaces de control y se compara con políticas nativas de referencia. `performance` mantiene una orientación hacia niveles altos de rendimiento, `ondemand` responde a la carga observada y `schedutil` incorpora información del planificador en la selección de frecuencia [21, 29].

Una solicitud de frecuencia no equivale necesariamente al reloj que termina usando el procesador. Mecanismos como `intel_pstate`, HWP o Turbo Boost pueden intervenir en el punto operativo efectivo según las condiciones de ejecución [30, 21]. Esta diferencia obliga a distinguir entre la acción solicitada y el estado alcanzado, ya que la comparación energética solo tiene sentido si el cambio atribuido a DVFS ocurrió realmente. El mismo principio aplica al tiempo de transición. Una decisión cuyo efecto tarda en materializarse puede perder utilidad cuando la fase de la carga cambia antes de que el nuevo nivel llegue a aprovecharse.

En GPU, la relación entre frecuencia y rendimiento también depende del recurso dominante. El dominio de procesamiento asociado a los SM y el dominio de memoria no res-

ponden de la misma manera ante una variación del reloj [20, 2]. Una carga *compute-bound* suele mostrar mayor sensibilidad a la capacidad de procesamiento, mientras que en una carga *memory-bound* elevar el reloj de los SM puede aportar una mejora menor si el movimiento de datos sigue imponiendo el límite. El régimen inferido adquiere sentido para el control precisamente porque anticipa esa sensibilidad y permite evitar cambios que añaden consumo sin una reducción proporcional del tiempo.

El cambio de reloj en el acelerador tampoco es instantáneo. Entre la solicitud y la estabilización del nuevo punto operativo existe una latencia que depende del dispositivo y de la transición realizada [43]. Si ese tiempo es comparable con la duración de la fase sobre la que se pretende actuar, el beneficio potencial se reduce porque parte del intervalo transcurre antes de alcanzar el estado deseado. La actuación DVFS en GPU debe interpretarse, por ello, junto con la escala temporal de las fases y no únicamente a partir de la clase inferida.

1.1.5. Cargas de trabajo y calibración

Las aplicaciones HPC combinan regiones con demandas diferentes sobre cómputo y memoria, de modo que una medida agregada de toda la ejecución puede ocultar cambios internos de comportamiento. Los kernels permiten aislar unidades de trabajo con un patrón computacional definido, mientras que los benchmarks aportan implementaciones reproducibles con diversidad algorítmica suficiente para observar cómo responde la plataforma [46, 19]. Su procedencia no determina la clase. Una carga se considera *compute-bound* o *memory-bound* a partir de la intensidad operacional medida respecto al *ridge point*, no porque la literatura o el nombre del benchmark la describan de esa manera.

Los microbenchmarks cumplen una función distinta. Al reducir la ejecución a operaciones controladas, permiten estimar propiedades de la plataforma como el ancho de banda de memoria y el rendimiento aritmético [19]. Estas mediciones sirven para construir los techos de Roofline y calibrar la referencia con la que luego se etiquetan las cargas de caracterización. El conjunto de entrenamiento se apoya así en benchmarks diversos, mientras que los microbenchmarks proporcionan los límites físicos con los que se interpreta su comportamiento.

1.1.6. Clasificación supervisada ligera

La inferencia del régimen se plantea como un problema de clasificación supervisada en el que una etiqueta conocida durante el entrenamiento se relaciona con señales que también puedan observarse durante la operación del sistema [25]. En este caso, la etiqueta procede de Roofline y las entradas corresponden a telemetría de bajo costo. La intensidad operacional y

las magnitudes que revelan directamente la regla utilizada para construir esa etiqueta deben permanecer fuera del vector de características, pues incluirlas convertiría la clasificación en una repetición del criterio de etiquetado y no en una inferencia útil para el lazo de control.

El carácter ligero del modelo se define por su costo de operación y no por el nombre del algoritmo. La regresión logística ofrece una referencia lineal, mientras que los árboles de decisión representan relaciones no lineales mediante reglas de partición sencillas [25]. Random Forest y Extra Trees combinan múltiples árboles para reducir la dependencia de una sola partición, a cambio de evaluar varios estimadores durante la inferencia [8, 16]. XGBoost construye el ensamble de forma secuencial y permite representar interacciones más complejas entre características [12]. Comparar estas familias permite determinar si el aumento de capacidad predictiva justifica también el costo temporal que introduce en una decisión en línea.

La evaluación debe reflejar tanto el desempeño global como el comportamiento sobre cada clase. La matriz de confusión permite observar cómo se distribuyen los errores entre *compute-bound* y *memory-bound*, mientras que el F_1 combina precisión y sensibilidad y su promedio macro otorga el mismo peso a ambas clases [40]. Esta propiedad resulta útil cuando la distribución de etiquetas no es uniforme, ya que evita que la clase más frecuente determine por sí sola la lectura del resultado. Una línea base mayoritaria proporciona además una referencia mínima para comprobar si las señales de entrada contienen información discriminativa bajo el protocolo de evaluación.

La latencia de inferencia forma parte de esa evaluación porque cada predicción consume tiempo dentro del ciclo de control. Un valor medio puede ocultar ejecuciones ocasionalmente más lentas, por lo que percentiles como p95 y p99 permiten describir la cola de la distribución sin presentarla como un peor caso absoluto [15]. La elección de un modelo no puede separarse por completo de este costo. Una mejora predictiva deja de ser atractiva si la inferencia consume una fracción significativa del intervalo en el que todavía puede aplicarse la decisión.

1.1.7. Optimización de hiperparámetros y Optuna

Además de los parámetros aprendidos durante el entrenamiento, cada familia de modelos contiene hiperparámetros que condicionan su estructura o su proceso de ajuste. La profundidad de un árbol, el número de estimadores de un ensamble o la intensidad de regularización son ejemplos de valores que deben seleccionarse antes de obtener el modelo final [7]. Su elección modifica tanto la capacidad de ajuste como la complejidad resultante, de modo

que comparar algoritmos sin considerar sus configuraciones puede confundir las propiedades de una familia con una elección particular de hiperparámetros.

Optuna organiza esta búsqueda mediante ensayos sucesivos o *trials*, donde cada configuración se evalúa a partir de una función objetivo y los resultados obtenidos orientan las propuestas posteriores [1]. Entre los métodos disponibles se encuentra el *Tree-structured Parzen Estimator*, que utiliza los ensayos ya observados para modelar regiones asociadas con mejores y peores resultados y concentrar parte de la exploración donde existe mayor expectativa de mejora. La finalidad no es recorrer de forma exhaustiva todas las combinaciones, sino asignar el presupuesto de evaluación de manera más informada.

La optimización introduce, al mismo tiempo, un riesgo de adaptación a los datos usados para escoger la configuración. Cuando la misma información participa en la selección y en la estimación final del desempeño, la evaluación puede quedar favorecida por el propio proceso de búsqueda. La validación anidada separa ambas funciones mediante un nivel interno para ajustar hiperparámetros y otro externo para estimar generalización [10]. Si varias observaciones proceden de una misma familia algorítmica, esa relación también debe preservarse durante la partición para evitar que variantes estrechamente relacionadas aparezcan a ambos lados de la evaluación [38].

Esta separación estadística no corrige una fuga presente en las características. Una variable derivada de la etiqueta, o suficientemente cercana a la regla que la construye, puede ofrecer al modelo información que no estaría disponible en una predicción real [26]. En una clasificación basada en Roofline, la distinción entre la fuente de verdad y la fuente de inferencia debe mantenerse desde el diseño del conjunto de datos. Optuna puede encontrar una configuración eficiente dentro del espacio propuesto, pero no puede convertir en válida una entrada que ya contiene la respuesta que el modelo intenta aprender.

1.1.8. Producto Energía–Retardo

La reducción de potencia no implica necesariamente una mejora energética si el tiempo de ejecución aumenta en una proporción suficiente para compensarla. Por esta razón, la evaluación de DVFS requiere observar ambas magnitudes de forma conjunta. El Producto Energía–Retardo, o EDP, expresa este compromiso mediante [28]:

$$EDP = E \cdot T \tag{1.5}$$

E representa la energía consumida y T el tiempo de ejecución. Un valor menor indica un

compromiso más favorable bajo este criterio, pero la formulación puede modificarse cuando se desea penalizar con mayor fuerza la degradación temporal:

$$ED^n P = E \cdot T^n \quad (1.6)$$

Con $n = 1$ se recupera el EDP clásico y con $n = 2$ se obtiene ED^2P , donde el tiempo adquiere mayor peso [28, 2]. La elección del exponente modifica la forma en que se ordenan las configuraciones y debe mantenerse explícita al comparar políticas. Para Hyperion, esta familia de métricas permite valorar si una reducción energética justifica el cambio de tiempo producido por la actuación DVFS, evitando interpretar potencia, energía o rendimiento de manera aislada.

1.2. Estado del Arte

La gestión energética en sistemas de alto rendimiento ha pasado de aplicar reducciones de frecuencia de forma relativamente uniforme a considerar con mayor detalle el comportamiento de cada carga. Este cambio responde a una limitación conocida de DVFS. Reducir el punto operativo puede disminuir la potencia, pero el resultado energético depende de cuánto se prolongue la ejecución. Los trabajos más próximos a esta investigación pueden leerse, por tanto, como una evolución desde la caracterización de esa sensibilidad hasta mecanismos capaces de reconocer fases y actuar durante la ejecución.

Calore et al. [9] analizaron el efecto de DVFS sobre una CPU Intel Haswell y una GPU NVIDIA K80 utilizando kernels y una aplicación HPC completa. Al modificar la frecuencia en regiones de ejecución observaron que una misma configuración no ofrece el mismo compromiso entre energía y tiempo para toda la aplicación. El trabajo establece una relación directa entre el comportamiento local de la carga y la conveniencia de cambiar el punto operativo, aunque la actuación depende de conocer previamente las regiones donde debe aplicarse. El problema queda entonces planteado de una forma que resulta relevante para Hyperion. Si las fases responden de manera distinta a DVFS, el control necesita reconocer esas diferencias sin depender de anotaciones específicas en cada aplicación.

Guerreiro et al. [20] abordaron esa necesidad desde la caracterización de aplicaciones GPGPU. Su propuesta recoge métricas en una configuración inicial y utiliza esa información para estimar cómo responderá la carga ante otros estados de frecuencia de cómputo y memoria. Con ello reducen la necesidad de recorrer exhaustivamente el espacio DVFS y muestran que la sensibilidad de una aplicación puede anticiparse a partir de señales del hardware. El

enfoque, sin embargo, se concentra en caracterizar la respuesta de una carga y no en seguir de manera continua las transiciones entre fases *compute-bound* y *memory-bound*.

Ali et al. [2] avanzaron en una dirección similar mediante modelos de potencia y tiempo contruidos a partir de métricas obtenidas en la configuración predeterminada de la GPU. A partir de esas estimaciones seleccionan frecuencias sin ejecutar cada alternativa y consideran de forma conjunta el ahorro energético y la degradación del desempeño. En las plataformas evaluadas reportaron reducciones promedio de energía de 29.6 % en NVIDIA GA100 y 22.6 % en GV100, con pérdidas de rendimiento de 5.2 % y 4.7 %, respectivamente. El resultado refuerza la idea de que una caracterización limitada puede ser suficiente para orientar la selección de frecuencia, aunque nuevamente la unidad de decisión es la carga caracterizada y no una sucesión de fases detectadas durante la ejecución.

La identificación automática del régimen aparece de manera más directa en MCBound, propuesto por Antici et al. [3]. El trabajo formula la separación entre cargas *memory-bound* y *compute-bound* como un problema de clasificación a partir de datos de ejecuciones HPC. Sobre información del supercomputador Fugaku, los autores reportaron un F_1 macro de al menos 0.89, lo que muestra que el régimen puede inferirse mediante señales observables sin depender exclusivamente de una clasificación manual. MCBound se aproxima al problema predictivo de Hyperion, pero utiliza la clase para caracterizar trabajos y no como entrada directa de una política que modifique la frecuencia de CPU o GPU durante la ejecución.

Simsek et al. [39] estudiaron precisamente la actuación por regiones en simulaciones astrofísicas ejecutadas sobre GPU. La instrumentación de SPH-EXA les permitió modificar la frecuencia en puntos determinados y reducir el consumo energético por GPU hasta en 7.82 %, con una degradación de rendimiento cercana a 2.95 %. El resultado confirma que adaptar la frecuencia dentro de una misma aplicación puede ser más conveniente que mantener un estado uniforme, aunque la política continúa ligada a regiones conocidas del código. La diferencia frente a una estrategia basada en telemetría es importante, porque en esta última la frontera entre comportamientos debe detectarse a partir de lo que el hardware expone durante la ejecución.

DRLCAP, presentado por Wang et al. [44], lleva esta idea hacia un control *online* basado en aprendizaje por refuerzo profundo. El marco detecta cambios de fase y ajusta límites de frecuencia de GPU durante la ejecución, con una reducción media de energía del 22 % y menos de 3 % de degradación del rendimiento en las arquitecturas estudiadas. Este antecedente demuestra que una política aprendida puede intervenir en tiempo real, pero adopta una solución más compleja y centrada en GPU. Hyperion parte de otra restricción. La inferencia

debe mantenerse ligera y la política debe poder operar sobre un entorno heterogéneo donde CPU y GPU no comparten las mismas fuentes de telemetría ni los mismos mecanismos de actuación.

La literatura revisada cubre, por separado, los componentes principales del problema. Calore et al. muestran que la respuesta a DVFS cambia entre regiones, Guerreiro et al. y Ali et al. relacionan telemetría con sensibilidad y selección de frecuencia, MCBound demuestra que el régimen puede inferirse mediante clasificación y los trabajos de Simsek et al. y Wang et al. muestran distintas formas de actuación durante la ejecución. La separación entre estas contribuciones deja espacio para estudiar una integración donde la clase inferida funcione como contexto de una política DVFS y donde el costo de observar, clasificar y actuar forme parte de la evaluación.

Esta integración también exige tratar CPU y GPU como dispositivos distintos y no como dos instancias del mismo problema. La mayoría de los trabajos recientes de predicción y control se concentra en GPU, mientras que Calore et al. ofrecen una de las referencias que consideran ambos tipos de dispositivo [9]. En CPU, las PMU y los contadores *uncore* permiten observar señales que no tienen un equivalente directo en NVML. En GPU, el perfilado detallado necesario para construir la referencia Roofline tiene un costo incompatible con la inferencia ligera y debe separarse de la telemetría utilizada durante el control. El aporte que se investiga en Hyperion se sitúa en esa diferencia de observabilidad, utilizando una referencia física común para definir el régimen, pero aceptando que las señales con las que se intenta inferirlo y los mecanismos de frecuencia disponibles son distintos en cada dispositivo.

Capítulo 2

Metodología

2.1. Enfoque metodológico

El enfoque metodológico fue cuantitativo, experimental y aplicado. La variable de respuesta fue el Producto Energía–Retardo (EDP), calculado a partir de la energía consumida y el tiempo de ejecución. Para analizar los resultados e identificar el origen de las variaciones, también se consideraron estas dos magnitudes por separado.

La variable independiente fue el estado de frecuencia, que se modificó para un conjunto controlado de cargas de trabajo. Sus efectos se evaluaron mediante el tiempo de ejecución, la energía y el EDP. Además, se desarrolló un agente de software y se comparó su desempeño con el de las políticas de frecuencia proporcionadas por el sistema operativo.

El estudio se desarrolló en cuatro fases: caracterización de las cargas y recolección de telemetría; desarrollo del clasificador; implementación del agente; y validación experimental del sistema.

2.2. Diseño metodológico

2.2.1. Población y muestra

La población de estudio comprendió las aplicaciones ejecutables en sistemas heterogéneos CPU–GPU. De esta población se seleccionó una muestra no probabilística e intencional de *benchmarks* y microbenchmarks con regímenes de ejecución contrastantes. Estas cargas permitieron construir un conjunto de datos controlado y reproducible para el entrenamiento, sin representar de forma exhaustiva todas las aplicaciones científicas.

Las cargas se incluyeron en el catálogo cuando verificaron la corrección de sus resultados, permitieron medir el trabajo de punto flotante y produjeron una magnitud comparable con un techo Roofline expresado en FLOPs. En CPU, el trabajo se midió mediante los contadores de instrucciones de punto flotante retiradas descritos en la sección 2.3.11; en GPU se obtuvo con el perfilado presentado en la sección 2.3.5. Cuando una carga reportó su propio conteo de operaciones, este se utilizó como contraste independiente. Las cargas dominadas por aritmética entera se excluyeron porque su intensidad operacional no era comparable con el techo empleado, de acuerdo con la regla de validez de la sección 2.3.2.

2.2.2. Plataforma experimental

Los experimentos se realizaron en el nodo heterogéneo descrito en la Tabla 2.1. La plataforma integró una GPU NVIDIA observable mediante NVML y permitió acceder a la medición energética interna del procesador sin privilegios administrativos. Estas capacidades permitieron recolectar telemetría de CPU y GPU bajo las mismas condiciones experimentales.

La codificación de algunos eventos crudos de la PMU dependió de la microarquitectura y no contó con un mapeo genérico en el kernel del nodo (sección ??). Antes de habilitar estos eventos, el instrumento verificó que la familia y el modelo del procesador coincidieran con los valores de la Tabla 2.1. Cuando la identificación no coincidió, la codificación se rechazó para evitar una interpretación incorrecta de los conteos. Esta comprobación incluyó los subeventos de `FP_ARITH_INST_RETIRED` utilizados para medir FLOPs (sección 2.3.11).

Tabla 2.1: Características de la plataforma experimental.

Componente	Descripción
Procesador	2 × Intel Xeon Gold 5315Y
Microarquitectura	Ice Lake-SP (10 nm, generación de servidor lanzada en 2021)
Identificador CUID (cpu family/model)	6 / 106
Extensiones SIMD relevantes	AVX2, FMA3, AVX-512 (avx512f, avx512bw, avx512cd, avx512dq, avx512vl, avx512_vnni, entre otras)
Núcleos	8 núcleos físicos por zócalo, 2 hilos por núcleo (32 lógicos)
Nodos NUMA	2 (uno por zócalo)
Jerarquía de caché	L1d 48 KiB, L1i 32 KiB, L2 1.25 MiB por núcleo; L3 12 MiB por zócalo
Tamaño de línea de caché	64 bytes
Controlador de frecuencia	intel_pstate
Rango de frecuencia	0.8–3.6 GHz
Dominio de frecuencia	Por núcleo lógico
Medición energética	Intel RAPL (dominios de paquete y DRAM por zócalo)
Acelerador	NVIDIA A100 PCIe 40 GB
Gestor de recursos	Slurm

Las condiciones de ejecución se controlaron mediante una asignación exclusiva del nodo a través del gestor de recursos. Los procesadores lógicos utilizados se restringieron a los hilos primarios de un mismo zócalo. Asimismo, no se asignaron de forma simultánea hilos hermanos del mismo núcleo físico, según lo expuesto en la sección 1.1.4.

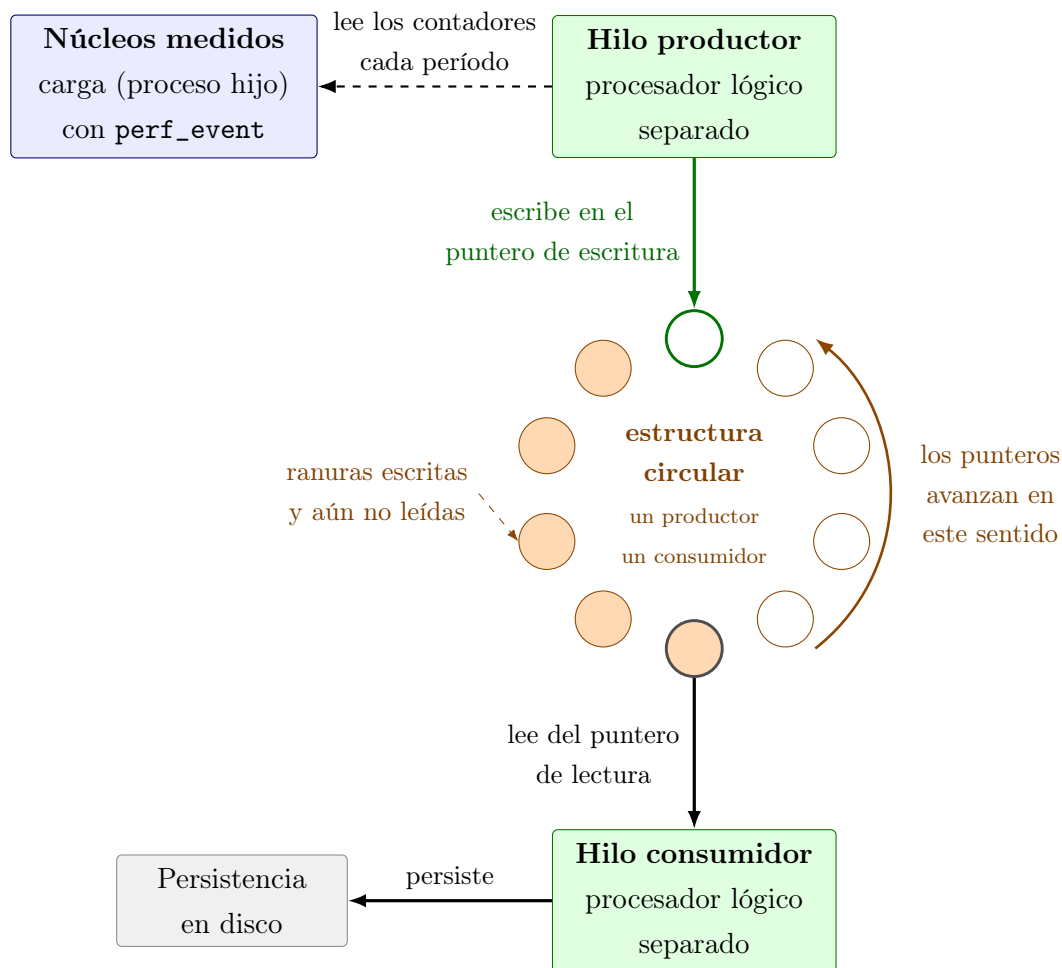
2.2.3. Instrumentos de recolección

La recolección de telemetría se realizó mediante un instrumento propio compuesto por dos capas con responsabilidades separadas.

La capa inferior fue un ejecutor escrito en C++ que inició la carga como proceso hijo y le adjuntó los contadores de hardware. Para cumplir los requisitos de atribución de la sección 1.1.3, el proceso hijo se detuvo antes de ejecutar su primera instrucción. Luego, el proceso padre abrió los contadores asociados a su identificador con la herencia habilitada y reanudó la ejecución. Con este protocolo se midió el árbol de procesos e hilos generado por la carga

durante todo el intervalo de observación, sin incluir actividad de otros procesos.

El muestreo se realizó con un hilo productor fijado a un procesador lógico distinto de los asignados a la carga. Este hilo leyó periódicamente los contadores y depositó las muestras en una estructura circular con un único productor y un único consumidor. El hilo consumidor, fijado a otro procesador lógico, extrajo y persistió las muestras. Esta separación redujo la interferencia de la escritura en disco sobre la periodicidad del muestreo. Además, las pérdidas de la estructura circular se registraron como indicador de integridad de cada corrida. La Figura 2.1 muestra el recorrido de las muestras.



si el puntero de escritura alcanza al de lectura la muestra se pierde;
ese conteo de pérdidas es el indicador de integridad de la corrida

Figura 2.1: Ruta de una muestra de telemetría, desde la lectura de los contadores hasta su persistencia. El hilo productor escribe cada muestra en la ranura que señala el puntero de escritura (borde verde) y el hilo consumidor la retira por el puntero de lectura (borde negro); ambos punteros avanzan en el mismo sentido, de modo que las ranuras naranjas son las escritas que el consumidor aún no ha leído. Los dos hilos se fijan a procesadores lógicos distintos de los que ejecutan la carga, y la estructura circular desacopla la cadencia de muestreo de la escritura a disco.

La capa superior fue un orquestador escrito en Python que interpretó el manifiesto de campaña, detectó las capacidades del nodo, ejecutó las verificaciones previas y planificó cada combinación experimental. Además, aplicó el estado de frecuencia, validó las corridas y transformó las muestras crudas en el conjunto de datos final. La capa C++ realizó la medición sensible a la latencia, mientras que la capa Python administró la planificación y la

trazabilidad del experimento. La Figura 2.2 muestra el recorrido de una campaña.

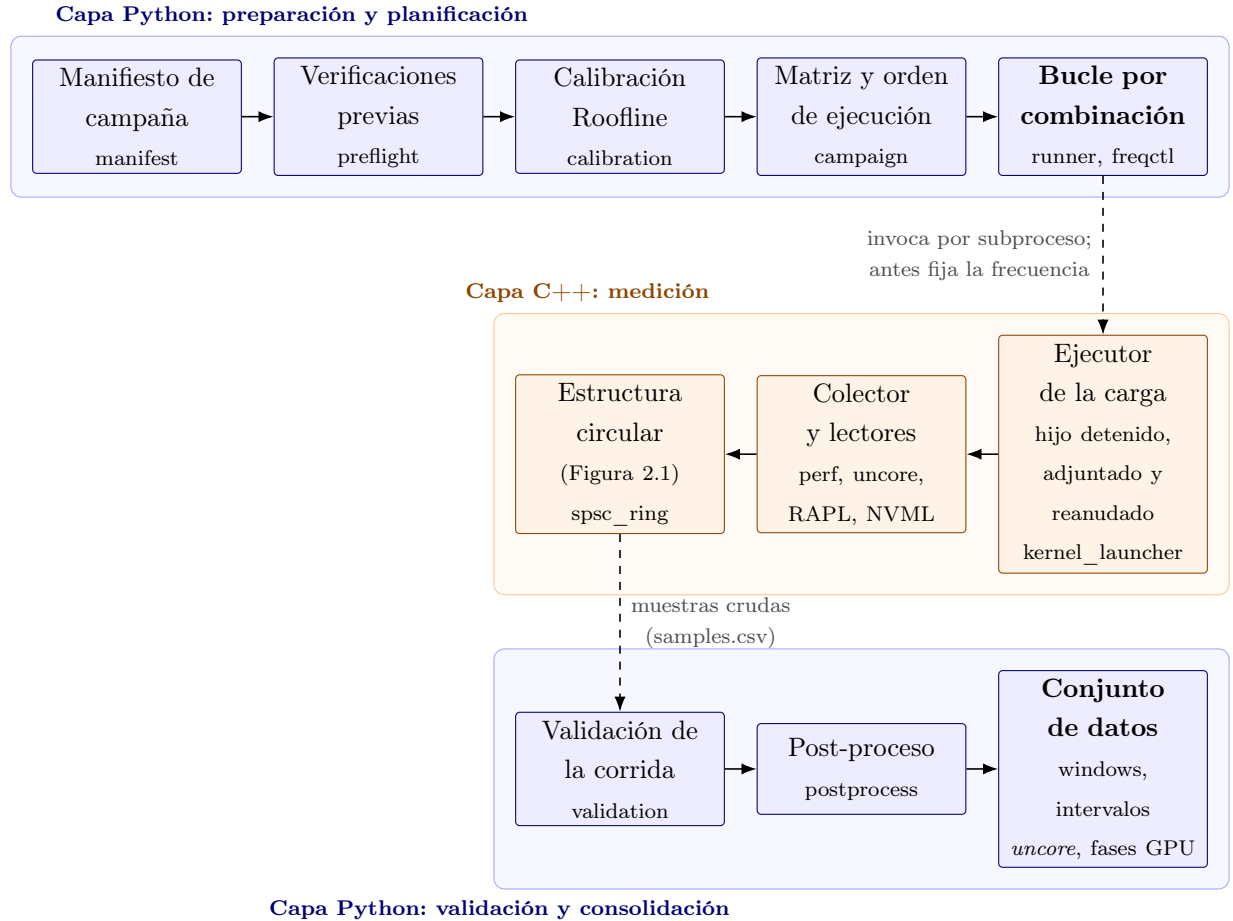


Figura 2.2: Recorrido de una campaña a través de las dos capas del instrumento. La capa Python prepara la campaña (banda superior), invoca por subproceso el ejecutor de la capa C++ una vez por combinación de carga y frecuencia (banda intermedia), y consolida las muestras crudas que este produce en el conjunto de datos (banda inferior). La capa C++ concentra la medición sensible a la latencia y la capa Python administra la lógica experimental.

2.2.4. Variables observadas

Una ventana de muestreo fue el intervalo entre dos lecturas consecutivas de los contadores. Como cada lectura contenía valores acumulativos, las métricas del intervalo se calcularon mediante la diferencia entre la lectura actual y la anterior, según la sección 1.1.3. La primera lectura de cada corrida no produjo una ventana porque no tuvo un valor previo; esta condición se registró como “sin diferencia previa” en la Tabla 2.6. La duración se obtuvo a partir del tiempo efectivo entre ambas lecturas, medido con un reloj monótono, y no mediante el período nominal configurado.

La Tabla 2.2 enumera las señales activas del instrumento. Distingue los contadores de núcleo, atribuibles a la carga, de los contadores del controlador de memoria, cuyo alcance correspondió al zócalo. El número de eventos se ajustó al presupuesto de contadores físicos simultáneos, verificado en la plataforma para evitar la multiplexación descrita en la sección ??.

Tabla 2.2: Señales de telemetría de CPU activas en el instrumento de producción.

Señal exportada	Evento e interpretación
Contadores de núcleo (<i>atribuibles al proceso medido</i>)	
<code>instructions</code>	<code>PERF_COUNT_HW_INSTRUCTIONS</code> . Base de IPC e instrucciones por segundo.
<code>cycles</code>	<code>PERF_COUNT_HW_CPU_CYCLES</code> . Denominador de IPC y de las fracciones de <i>stall</i> .
<code>cache_references</code> , <code>cache_misses</code>	Eventos genéricos de <code>perf</code> que producen <code>cache_miss_rate</code> y MPKI, los cuales indican presión de memoria, no bytes movidos a DRAM.
<code>stalled_cycles_</code> <code>mem_any</code>	Evento crudo <code>STALLS_MEM_ANY</code> (0xA3/0x14, cmask=0x14). Ciclos de ejecución bloqueados mientras hay una carga de memoria pendiente; se normaliza como <code>stall_mem_ratio</code> . Señal predictiva, no la etiqueta Roofline.
<code>fp_*_double</code>	Subeventos de doble precisión de <code>FP_ARITH_INST_RETIRED</code> (0xC7). Numerador de FLOPs, ponderados por 1, 2, 4 u 8 elementos.
<code>time_enabled_ns</code> , <code>time_running_ns</code>	Metadatos de <code>perf</code> . Diagnostican la multiplexación y escalan los conteos; las ventanas degradadas se excluyen del entrenamiento.
Ámbito de zócalo o de sistema (<i>no atribuibles a un proceso</i>)	
Comandos CAS del IMC: <code>read/write_interval</code>	Número de comandos <i>Column Address Strobe</i> (CAS) de lectura y de escritura emitidos por el controlador de memoria integrado (IMC), contabilizados por su PMU de <i>uncore</i> . Cada comando transfiere una línea de 64 bytes hacia o desde la DRAM, de modo que la suma de ambos es el denominador DRAM del modelo Roofline. Su intervalo práctico (≥ 10 ms) es más grueso que la ventana de CPU, y su etiqueta se difunde a las ventanas que cubre.
RAPL: <code>pkg_uj</code> , <code>dram_uj</code>	Energía acumulada por zócalo. Sus diferencias, con corrección de desbordamiento, producen energía y potencia; no intervienen en la etiqueta <i>compute/memory-bound</i> .
<code>scaling_cur_freq</code> por CPU delegado	Lectura de <i>sysfs</i> por núcleo lógico. Verifica la frecuencia efectiva bajo carga y detecta divergencias entre los núcleos delegados (sección 1.1.4).

A partir de estas señales se calcularon, para cada ventana, el IPC, la tasa de fallos de caché, el MPKI, `stall_mem_ratio`, la intensidad operacional y las magnitudes energéticas. Las tasas de núcleo utilizaron el intervalo real entre lecturas consecutivas; la intensidad y la etiqueta Roofline conservaron la granularidad del intervalo *uncore*, según la sección 1.1.3.

2.3. Fase 1: Recolección de información y caracterización de cargas

La primera fase tuvo como objetivo construir el conjunto de datos etiquetado empleado para entrenar el clasificador, junto con el instrumento y el protocolo necesarios para asegurar la validez de las mediciones. A continuación se presenta el procedimiento aplicado.

2.3.1. Verificación previa de la plataforma

Antes de cada campaña se ejecutaron verificaciones automáticas para confirmar que el nodo cumpliera las condiciones del diseño experimental. Se comprobaron las capacidades de control y medición de frecuencia, la afinidad NUMA y la configuración de multihilo, la disponibilidad de contadores y dominios energéticos, la integridad y los permisos de los binarios, la memoria y el almacenamiento disponibles y, cuando correspondía, los permisos de escritura sobre los dominios de frecuencia asignados.

Las comprobaciones se clasificaron en obligatorias e informativas; el incumplimiento de una condición obligatoria detuvo la campaña antes de modificar el sistema o producir datos. Este control formó parte del mismo comando de ejecución, por lo que ninguna medición pudo iniciarse sin completar previamente la verificación del entorno.

2.3.2. Catálogo de cargas de trabajo

El catálogo se organizó en cargas de calibración y cargas de caracterización. Para estimar los techos del modelo Roofline se utilizaron un microbenchmark de ancho de banda, cuyo conjunto de trabajo excedió la caché de último nivel, y otro de alta densidad aritmética; con ellos se obtuvieron BW_{pico} y P_{pico} , respectivamente. Las cargas de caracterización procedieron de suites de uso extendido y se emplearon sin modificar su código fuente. Cada binario se compiló en la plataforma experimental y se identificó mediante una suma de verificación, comprobada antes de cada corrida para evitar que una recompilación alterara las condiciones de la campaña. La Tabla 2.3 presenta las cargas y su procedencia.

Tabla 2.3: Catálogo de cargas de trabajo.

Carga	Suite u origen	Papel y régimen esperado
<i>Calibración de los techos Roofline</i>		
STREAM	McCalpin [34]	BW_{pico} de CPU.
ERT	Lo et al. [33]	P_{pico} de CPU.
Microbenchmarks propios (BW, FP32, FP64)	Elaboración propia	Techos de ancho de banda y de cómputo del acelerador.
DGEMM	NVIDIA cuBLAS	Referencia independiente del techo de cómputo del acelerador.
<i>Conjunto de datos — CPU</i>		
BT, MG, CG, SP, FT, LU	NAS Parallel Benchmarks [4]	Métodos implícitos, multigrid, gradiente conjugado y FFT; mayoría <i>memory-bound</i> .
DGEMM	Biblioteca BLAS optimizada [18]	Multiplicación densa de matrices; ancla <i>compute-bound</i> .
LavaMD (OpenMP)	Rodinia [11]	Dinámica molecular de N-cuerpos; régimen intermedio.
3MM (OpenMP)	RAJA Performance Suite [6]	Triple multiplicación densa, con verificación numérica propia; régimen intermedio.
<i>Conjunto de datos — acelerador</i>		
gaussian, backprop, heartwall, LUD, myocyte, DWT2D	Rodinia [11]	Eliminación gaussiana, retropropagación, seguimiento de rasgos, factorización LU, electrofisiología y transformada wavelet; mayoría <i>memory-bound</i> .
LavaMD	Rodinia [11]	Principal ancla <i>compute-bound</i> del acelerador.
DGEMM	NVIDIA cuBLAS	Referencia <i>compute-bound</i> del acelerador.
<i>Candidatos de ampliación del catálogo (sección 2.3.3)</i>		
EP	NAS Parallel Benchmarks [4]	Ancla <i>compute-bound</i> pura en CPU.
kmeans, SRAD, NW, particlefilter	Rodinia [11]	Patrones de acceso irregular en CPU: agrupamiento iterativo, difusión anisotrópica, programación dinámica en diagonal y filtro de partículas.
kmeans, SRAD, NW, b+tree, CFD	Rodinia [11]	Los mismos patrones en el acelerador, más recorrido de árbol y volúmenes finitos; kmeans , SRAD y NW forman pares CPU/acelerador.

Las cargas de calibración y caracterización iniciales se utilizaron para construir y validar el instrumento; posteriormente, el catálogo se amplió con nuevas cargas y se definió la familia algorítmica como unidad de diversidad.

Una familia algorítmica reunió las variantes de tamaño que compartían el mismo patrón de cómputo y acceso a memoria. Esta agrupación evitó considerar como ejemplos independientes varias ejecuciones del mismo algoritmo y permitió evaluar la generalización sobre patrones computacionales distintos.

La admisión de familias al conjunto definitivo siguió tres reglas:

1. **Representación por clase y dispositivo.** Cada clase debía incluir varias familias distintas en CPU y GPU; las variantes de tamaño de una misma familia no aumentaron ese conteo.
2. **Separación entre controles y datos de entrenamiento.** Los microbenchmarks propios se reservaron para calibrar y verificar el instrumento, por lo que no se utilizaron para entrenar el clasificador.
3. **Validez de la medida de trabajo.** Solo se admitieron cargas cuyo trabajo pudiera expresarse de manera significativa mediante intensidad operacional en FLOPs por byte; las cargas de aritmética entera o con una semántica incompatible se excluyeron y se registró el motivo.

Las familias adicionales se eligieron para ampliar la diversidad de patrones de acceso a memoria y la representación de ambas clases. Antes de incorporarlas se exigieron un binario reproducible, una suma de verificación, una validación numérica y, para GPU, una intensidad operacional obtenida mediante el procedimiento de la sección 2.3.5.

2.3.3. Selección preliminar y definición del catálogo definitivo

Antes de ejecutar el barrido completo se realizó una evaluación preliminar con tres niveles de frecuencia por dispositivo y el número mínimo de repeticiones. Esta etapa permitió comprobar la calidad de las señales, descartar cargas no utilizables y estimar la cobertura de las clases, sin buscar configuraciones óptimas de energía. En GPU, la intensidad operacional se caracterizó previamente mediante el procedimiento de la sección 2.3.5.

Para cada familia se analizaron su posición respecto al punto de inflexión, la representación de las clases y las causas de rechazo por calidad. En GPU, donde la etiqueta se obtuvo fuera de línea y permaneció constante durante cada corrida, también se examinó si las señales NVML permitían diferenciar los regímenes. Con estos criterios se definió y versionó la

lista de familias admitidas en la campaña definitiva; cuando la cobertura fue insuficiente, se incorporaron nuevas familias sin reducir el criterio mínimo establecido.

2.3.4. Calibración y criterio de etiquetado

La calibración se ejecutó al inicio de cada campaña con la misma asignación de núcleos y la misma afinidad utilizadas en las corridas posteriores. A partir de los valores medidos se calculó el punto de inflexión de la ecuación (1.3) para cada dispositivo, precisión aritmética y nivel de frecuencia, de acuerdo con la sección 1.1.2. Los resultados se compararon con las referencias empíricas declaradas en el manifiesto; una diferencia superior a la tolerancia establecida detuvo la campaña para evitar el uso de una calibración obtenida bajo condiciones anómalas.

La etiqueta de cada unidad de observación se obtuvo al comparar su intensidad operacional con el punto de inflexión calibrado, sin asignación manual. La unidad de observación utilizada para este cálculo se define en la sección 2.3.10. El catálogo también conservó una expectativa cualitativa basada en la literatura, pero esta se empleó solo para detectar posibles inconsistencias del instrumento y nunca para sustituir o corregir la etiqueta medida.

La estabilidad de cada campaña se comprobó mediante repeticiones de una carga de referencia. Una dispersión relativa superior al umbral establecido se registró como advertencia, pues indicaba que las condiciones de medición no permitían sostener comparaciones precisas entre corridas.

2.3.5. Caracterización de intensidad operacional y calibración Roo-fine en GPU

NVML proporcionó durante cada corrida la potencia, la utilización, el reloj, la temperatura y la energía de la GPU, pero no los FLOPs ni los bytes requeridos para calcular la intensidad operacional. Por ello, cada carga se caracterizó previamente y fuera de línea con NVIDIA Nsight Compute [37], que permitió medir por lanzamiento el tráfico hacia la memoria del dispositivo y las operaciones de punto flotante ejecutadas. Para una precisión aritmética determinada, la intensidad operacional se calculó como

$$I^{\text{GPU}} = \frac{\sum_{\ell=1}^L (N_{\ell}^{\text{add}} + N_{\ell}^{\text{mul}} + 2 N_{\ell}^{\text{fma}})}{\sum_{\ell=1}^L B_{\ell}}, \quad (2.1)$$

donde ℓ recorre L lanzamientos consecutivos del kernel; N_ℓ^{add} , N_ℓ^{mul} y N_ℓ^{fma} son los conteos de operaciones de suma, multiplicación y multiplicación–suma fusionada de punto flotante que el perfilador reporta para el lanzamiento ℓ , y B_ℓ es el total de bytes transferidos hacia o desde la memoria del dispositivo en ese lanzamiento. Cada multiplicación–suma fusionada contribuye con dos operaciones, según la misma convención adoptada para CPU en la ecuación (2.2).

La caracterización se repitió con cantidades crecientes de trabajo y la intensidad solo se aceptó cuando el cambio relativo entre los dos puntos de mayor tamaño fue inferior al 1 % y el número de lanzamientos observado coincidió con el solicitado. Las cargas que no cumplieron este criterio se registraron como no convergentes. Asimismo, se midieron simultáneamente las operaciones de precisión simple y doble; las cargas con mezcla no trivial de precisiones o sin trabajo de punto flotante significativo se excluyeron, porque no podían compararse de forma válida con un único techo Roofline. Para asegurar la trazabilidad se conservaron las salidas del perfilador, los comandos y las versiones del controlador y de las herramientas.

La intensidad aceptada se registró como propiedad de la carga y se reutilizó durante el procesamiento de la campaña NVML. Esta medición se realizó una sola vez por configuración del problema, ya que el perfilado detallado altera el tiempo de ejecución y no era adecuado para la adquisición en línea. Cuando Nsight Compute requirió más contadores de los disponibles en una pasada, repitió el lanzamiento completo por grupos de métricas, por lo que no se aplicó el escalado temporal utilizado por la multiplexación de CPU.

Los techos de GPU se calibraron por separado para precisión simple y doble mediante microbenchmarks de ancho de banda y de multiplicación–suma fusionada sobre datos residentes en registros. Esta separación evitó comparar una carga con un techo de distinta precisión o con rutas especializadas que no representaran la aritmética general del catálogo.

2.3.6. Dimensionamiento de las cargas GPU y verificación de invarianza de la intensidad operacional

Los parámetros de las cargas GPU se ajustaron para obtener una duración suficiente sin alterar innecesariamente la intensidad operacional medida. Cuando el parámetro solo repetía el mismo patrón de cómputo, se aumentó el número de repeticiones sin volver a caracterizar la carga; cuando modificaba el tamaño del problema, se repitió el procedimiento de la sección 2.3.5, pues el nuevo conjunto de datos podía cambiar la relación entre cómputo y tráfico de memoria.

Si la implementación no permitía repetir el mismo trabajo y el tamaño máximo estaba

limitado por el hardware, se conservó la mayor configuración válida y se registró la limitación de duración. No se forzaron configuraciones que invalidaran la caracterización previa.

2.3.7. Observabilidad efectiva y latencia de conmutación de reloj en GPU

Antes de la campaña GPU se midieron la cadencia efectiva de actualización de las señales NVML y la latencia de los cambios de reloj bajo carga. La cadencia se seleccionó comparando varios períodos de consulta y eligiendo el más amplio que conservara la cobertura de las cargas cortas y permitiera localizar los cambios de señal. Las lecturas consecutivas iguales no se trataron como observaciones independientes, porque la frecuencia de consulta puede ser mayor que la frecuencia de actualización del sensor.

La latencia de conmutación se midió para cada par de niveles que podía solicitar la política, con al menos tres repeticiones y la GPU bajo carga. El nivel de destino se consideró estable cuando varias lecturas consecutivas permanecieron dentro de la tolerancia definida y no se observaron condiciones de limitación del dispositivo. Como valor conservador se utilizó la mayor latencia observada; esta cota determinó el intervalo mínimo entre decisiones y permitió excluir del análisis energético las fases demasiado breves para completar una transición.

2.3.8. Matriz experimental y control de sesgos

La matriz experimental se construyó como el producto entre las cargas seleccionadas, los niveles de frecuencia y las repeticiones independientes. La evaluación preliminar de la sección 2.3.3 utilizó tres niveles por dispositivo para seleccionar las familias, mientras que la campaña definitiva empleó la rejilla de la Tabla 2.4. El nivel REF mantuvo la configuración nativa del sistema y se utilizó como referencia.

Tabla 2.4: Rejilla fina de frecuencia de la campaña definitiva.

Dispositivo	Niveles de reloj (MHz; REF es el punto nativo)
CPU	REF, 3200, 3100, 3000, 2900, 2800, 2600, 2400, 2200, 2000, 1400, 800
GPU	REF, 1410, 1350, 1290, 1230, 1170, 1110, 810, 510, 210

La separación entre niveles no fue uniforme: se concentraron más puntos en la parte alta del rango para observar si reducciones pequeñas de frecuencia disminuían el consumo

sin superar el presupuesto de degradación del rendimiento. En la parte baja se utilizaron intervalos mayores, suficientes para caracterizar la tendencia general.

Los niveles se declararon en MHz y se verificaron contra los relojes admitidos por el nodo. Cuando un valor no estaba disponible se utilizó el nivel soportado más cercano o se excluyó, dejando registro de la decisión. La calibración Roofline se repitió para cada dispositivo, precisión y nivel de frecuencia.

Para reducir el sesgo experimental, cada combinación se ejecutó en varias repeticiones independientes y el orden se aleatorizó mediante una semilla registrada. La afinidad, el número de hilos y la asignación de núcleos permanecieron constantes; además, el sistema restauró su configuración de frecuencia al finalizar cada campaña o ante una terminación anómala.

Para fijar las frecuencias de CPU se deshabilitó temporalmente *Turbo Boost*, ya que un límite escrito por núcleo no garantiza el reloj físico mientras la aceleración dinámica permanece activa. El estado se verificó por relectura y se restauró al finalizar; por esta razón, 3200 MHz fue el nivel fijo más alto y 3600 MHz quedó reservado al nivel nativo REF. Durante cada corrida se leyó la frecuencia de todos los núcleos asignados: la ausencia o inconsistencia estructural de las lecturas rechazó la corrida, mientras que las desviaciones puntuales respecto al objetivo se anotaron por ventana según la Tabla 2.5.

Tabla 2.5: Taxonomía de clasificación de frecuencia por ventana de muestreo.

Anotación	Condición
Válida	Todos los núcleos delegados leídos en la ventana caen dentro de la tolerancia declarada respecto al reloj efectivamente aplicado.
No confiable	Al menos un núcleo delegado se aparta de la tolerancia, fuera del margen de gracia posterior a una transición de nivel.
No verificable (margen de gracia)	La ventana cae dentro del margen de exclusión inmediatamente posterior a una transición de nivel, donde el transitorio físico de conmutación aún no se ha resuelto; la desviación, si la hay, no se atribuye a una falla de actuación.
No aplicable (gobernador nativo)	La corrida se ejecuta bajo el gobernador nativo de la plataforma (nivel de referencia), sin un objetivo de frecuencia fija contra el cual evaluar la tolerancia.

Esta anotación se conserva como columna independiente de la calidad general de la ventana descrita en la Tabla 2.6: una ventana puede ser válida en calidad general y, simultáneamente, no confiable en frecuencia, o viceversa. Solo las ventanas anotadas como válidas o no aplicables cuentan hacia el mínimo de ventanas exigido para aceptar la corrida; las anotadas como no confiables o no verificables se conservan íntegras en el conjunto de datos, con su anotación, pero no contribuyen a ese mínimo. Adicionalmente, cada corrida produce un resumen de cobertura de frecuencia — fracción de ventanas válidas y longitud de la racha más larga de ventanas no confiables consecutivas — que no condiciona por sí solo la aceptación, pero permite auditar después, corrida por corrida, cuánta de su duración total quedó efectivamente verificada en frecuencia.

2.3.9. Post-procesamiento y control de calidad de los datos

Las muestras crudas se transformaron en una traza auditable mediante la diferenciación entre lecturas consecutivas, conforme a la sección 1.1.3. Cada fila representó una ventana de muestreo e incluyó las métricas derivadas y los campos necesarios para reconstruir su procedencia, entre ellos el identificador de corrida, la carga, el nodo, la frecuencia solicitada y observada, las referencias de calibración y la suma de verificación del binario.

Ninguna observación se eliminó durante este procedimiento. Cada ventana se conservó con una anotación de calidad según la Tabla 2.6; de este modo, el filtrado se realizó durante el análisis y quedó documentada la proporción de datos en cada categoría.

El intervalo inicial de cada corrida se trató como un transitorio de arranque y se excluyó del análisis para caracterizar únicamente la fase estable del kernel. Su duración se calculó durante el post-procesamiento, por lo que la adquisición conservó la traza completa y permitió ajustar el criterio sin repetir la ejecución.

La detección se aplicó sobre instrucciones por ciclo en CPU y sobre la utilización del dispositivo en GPU, mediante ventanas móviles de tamaño fijo. Primero se buscó el instante a partir del cual el coeficiente de variación permaneció por debajo del umbral durante dos ventanas consecutivas y la media superó un nivel mínimo de actividad; esta última condición evitó confundir el reposo inicial de la GPU con un estado estable. Cuando este criterio no produjo un punto válido, se utilizó una detección de cambios basada en la reducción del error cuadrático entre segmentos y se seleccionó el primero cuya media alcanzó una fracción declarada de la meseta de mayor actividad [15, 5]. En las cargas de GPU, el límite propuesto también se contrastó con la transición observada en la traza de potencia; cuando la cadencia del dispositivo no permitió identificarla, la limitación quedó registrada en el catálogo y no se

asignó un valor sin respaldo.

La calibración se integró en la propia campaña. En una primera pasada se procesó la traza completa sin excluir ventanas iniciales; después se calcularon los intervalos de calentamiento, se actualizó el catálogo con control de versión y suma de verificación, y se reprocesaron los datos crudos sin volver a ejecutar los kernels. Finalmente, se reevaluó la aceptación de cada corrida con los intervalos calibrados y se registró cualquier cambio de veredicto, sin eliminar los artefactos originales.

Tabla 2.6: Taxonomía de anotaciones de calidad por ventana de muestreo.

Anotación	Condición
Válida	La ventana satisface todos los criterios de calidad.
Sin diferencia previa	Primera muestra de la corrida; carece de lectura anterior contra la cual diferenciar.
Excluida por calentamiento	La ventana pertenece al intervalo inicial declarado como transitorio de arranque.
Contadores degradados	Diferencia negativa, valor ausente, o fracción de tiempo contando inferior al umbral admitido.
Intensidad indefinida	No fue posible calcular la intensidad operacional, por ausencia de trabajo reportado o de tráfico observable.
Energía inválida	La lectura energética correspondiente no es válida en una campaña que la requiere.
Sin lectura de frecuencia	No se dispone de la frecuencia efectivamente observada durante la ventana.

Además, cada corrida se validó de forma integral mediante la estructura de muestreo, la suma de verificación del binario, el criterio de éxito de la carga y la unicidad de su identificador. Las corridas que no cumplieron estas condiciones se registraron como rechazadas y sus artefactos se conservaron para inspección.

2.3.10. Unidad de observación del conjunto de entrenamiento

Las ventanas se conservaron como registro auditable, pero no se trataron siempre como observaciones independientes para el entrenamiento. La unidad de análisis se definió por dispositivo, de acuerdo con la resolución de la medición que sustentó la etiqueta; así se evitó sobrestimar el tamaño efectivo del conjunto de datos y el desempeño del modelo.

CPU: intervalo del controlador de memoria. Cada fila correspondió a un intervalo de lectura del controlador de memoria, cuya resolución fue menor que la de los contadores de núcleo (sección 1.1.3). Los FLOPs y las demás métricas se recalcularon a partir de los deltas crudos agregados dentro de ese intervalo, en lugar de promediar cocientes calculados por ventana. Se rechazaron los intervalos sin lecturas válidas de memoria, con cobertura insuficiente, que cruzaron el límite de calentamiento o que contuvieron contadores o frecuencias degradados; el motivo quedó registrado y no se imputaron valores sustitutos.

GPU: corrida completa. Cada fila correspondió a una combinación de carga, frecuencia y repetición, porque las señales en línea se actualizaron a una cadencia menor y la intensidad operacional se midió una sola vez para todo el kernel (sección 2.3.7). Las características se obtuvieron mediante estadísticos robustos de las muestras válidas posteriores al calentamiento. La intensidad operacional, el punto de inflexión y la etiqueta se conservaron solo para trazabilidad y se excluyeron de las entradas del modelo; asimismo, los microbenchmarks con fases programadas se mantuvieron como controles diagnósticos y no se utilizaron para el entrenamiento.

2.3.11. Medición directa de FLOPs por ventana y su validación empírica

En CPU, los FLOPs de cada ventana se midieron directamente mediante los cuatro subeventos de doble precisión de la familia `FP_ARITH_INST_RETIRED`. Esta decisión evitó estimarlos a partir del total reportado por la carga y permitió alinear la medición con las demás señales de telemetría.

Codificación y ponderación. En el procesador Intel Xeon Gold 5315Y, la familia utilizó el *EventSelect* `0xC7` y las máscaras `0x01`, `0x04`, `0x10` y `0x40` para las operaciones escalares y vectoriales de 128, 256 y 512 bits. Como el sistema no expuso alias simbólicos para estos eventos, se programaron como eventos crudos mediante `perf_event_open`. Cada conteo se ponderó por el número de elementos FP64 de la instrucción; las operaciones FMA ya fueron contabilizadas por el hardware como dos operaciones por elemento y no requirieron una corrección adicional [23]:

$$\text{FLOPs}_i^{\text{HW}} = 1 \cdot \Delta N_{\text{escalar},i} + 2 \cdot \Delta N_{128,i} + 4 \cdot \Delta N_{256,i} + 8 \cdot \Delta N_{512,i} \quad (2.2)$$

La codificación se obtuvo de la definición oficial de Intel para Ice Lake-SP [23] y se

Tabla 2.7: FLOPs por instrucción retirada según ancho vectorial y tipo de operación, evento FP_ARITH_INST_RETIRED de Ice Lake-SP. La columna FMA ya está incorporada en el valor que reporta el contador de hardware.

Ancho vectorial	Elementos FP64	FLOPs (ADD/MUL)	FLOPs (FMA)
Escalar	1	1	2
128 bits	2	2	4
256 bits	4	4	8
512 bits	8	8	16

contrastó con el grupo FLOPS_DP de LIKWID [42].

Verificación. La medición se contrastó con una multiplicación de matrices de volumen analíticamente conocido y con un kernel multihilo de NAS Parallel Benchmarks, ambos ejecutados con la afinidad utilizada en la campaña. También se verificó la razón entre el tiempo de conteo y el tiempo habilitado para detectar multiplexación. La configuración de producción conservó los cuatro subeventos de punto flotante dentro del presupuesto de nueve contadores descrito en la sección ??; cuando alguno no estuvo disponible, la ventana quedó con intensidad operacional indefinida. Los resultados y el alcance de esta validación para Ice Lake-SP se presentan en la sección ??.

2.3.12. Reproducibilidad

Cada campaña se definió mediante un manifiesto bajo control de versiones, con las cargas, frecuencias, repeticiones, período de muestreo, asignación de núcleos, semilla y referencias de calibración. Junto con los datos se conservaron el perfil de capacidades del nodo, los resultados de calibración y los metadatos de cada corrida.

Para reanudar una campaña, el orquestador comparó la huella del protocolo almacenada con la del manifiesto vigente; cualquier diferencia detuvo la ejecución para evitar mezclar corridas obtenidas con configuraciones distintas. Cuando la huella coincidió, se reutilizó la calibración previamente validada. Además, después de modificar una carga se repitió una combinación completa y se comprobó la consistencia de su intensidad operacional, su punto de inflexión y su etiqueta.

2.3.13. Selección y registro de características

Las características se seleccionaron por separado para CPU y GPU. Primero se calcularon las correlaciones de Pearson y Spearman entre las columnas candidatas; cuando un par superó

el umbral definido, se conservó la medición física más directa. Después se utilizó el factor de inflación de la varianza para identificar relaciones entre más de dos variables, y se excluyeron las columnas ausentes, constantes o con valores no finitos.

Las variables empleadas para construir la etiqueta —intensidad operacional, punto de inflexión, FLOPs, bytes y expectativa cualitativa del catálogo— se prohibieron como entradas del clasificador, con el fin de evitar fuga de información (sección ??). La lista final se revisó y se registró en una especificación versionada antes del entrenamiento; también se documentaron las columnas descartadas y el motivo de cada decisión.

2.4. Fase 2: Desarrollo del modelo de aprendizaje automático

En esta fase se desarrollan las técnicas de análisis orientadas a la construcción de un modelo predictivo capaz de identificar el comportamiento de la carga de trabajo en tiempo de ejecución. El proceso de preprocesamiento incluye la limpieza, la normalización y la selección de características relevantes, partiendo del conjunto de datos producido en la Fase 1 y aplicando sobre él los criterios de filtrado a partir de las anotaciones de calidad descritas en la Tabla 2.6. Para GPU, este proceso ya se ejecutó (sección ??); para CPU permanece pendiente.

El problema se formula como una tarea de clasificación supervisada binaria, donde el modelo recibe como entrada un vector de telemetría — correspondiente a una ventana de ejecución en CPU, o a una corrida completa en GPU (sección ??) — y produce como salida la etiqueta de régimen. Dado que el sistema debe operar en tiempo de ejecución, se prioriza el uso de modelos de baja complejidad computacional, entre ellos árboles de decisión, bosques aleatorios, regresión logística y ensambles potenciados, comparados mediante métricas de clasificación y latencia de inferencia medida.

Un criterio metodológico relevante en esta fase es la estrategia de partición del conjunto de datos. Dado que las observaciones provenientes de una misma corrida no son independientes entre sí, una partición aleatoria a nivel de observación produciría una estimación optimista de la capacidad de generalización. Por ello, la partición se realiza a nivel de familia algorítmica (*leave-one-family-out* para GPU), de modo que la evaluación mida la capacidad del modelo para generalizar hacia algoritmos no observados. A partir de este análisis se selecciona el modelo que represente el mejor compromiso entre desempeño predictivo y costo de inferencia, y se serializa para su integración en el sistema de control.

2.5. Fase 3: Implementación del agente de control DVFS

2.5.1. Arquitectura de dos bucles desacoplados

El agente de control se implementó como un proceso en espacio de usuario formado por dos bucles desacoplados y una señal de coordinación. El bucle de CPU operó sobre ventanas de aproximadamente 1 ms, coherentes con la telemetría de la Fase 1, y ejecutó la inferencia y la decisión de frecuencia a baja latencia. El bucle de GPU operó por fases de ejecución completas, dado que la conmutación de reloj en GPU implicó un coste que no se amortizaba a la cadencia del orden del milisegundo. Cuando el bucle de GPU detectó actividad sostenida por encima de un umbral, forzó un piso de frecuencia en el dominio CPU para evitar interpretar esperas bloqueantes como fases *compute-bound*.

2.5.2. Detección de fase en GPU

La detección de fases en GPU se implementó mediante sondeo periódico de la utilización reportada por la interfaz de gestión del dispositivo, sin instrumentar el binario objetivo. Se configuró la cadencia de sondeo y un umbral de ruido para emitir eventos de inicio de fase al cruzar el umbral en sentido ascendente; las transiciones descendentes se registraron para trazabilidad. Esta solución se adoptó por su bajo nivel de intrusión y su eficacia práctica.

2.5.3. Diseño del bucle de CPU e integración con el clasificador

El bucle de CPU incorporó una máquina de decisión independiente de la biblioteca de inferencia, de modo que su lógica se verificó aisladamente. En cada ciclo se construyó el vector de características a partir del snapshot de telemetría, se invocó el modelo exportado desde la Fase 2 y se consultó la tabla de política. Solo se aplicó un nuevo nivel de frecuencia cuando la clase inferida cambió respecto al ciclo anterior, para reducir oscilaciones. La integración se realizó en el mismo lenguaje del colector de telemetría para evitar cruces costosos entre runtimes.

2.5.4. Derivación *offline* de la tabla de política

La tabla de política se derivó *offline* a partir del barrido de frecuencia de la Fase 1. Para cada combinación de dispositivo, clase de régimen y nivel de frecuencia se calculó la mediana del Producto Energía–Retardo por kernel y se comparó con la referencia nativa mediante pruebas de significancia pareadas (Wilcoxon o *t-test*, según corresponda). Cuando ningún nivel mejoró el EDP de forma significativa, la política se fijó en *no actuar*. Para GPU

se exigió además una latencia de conmutación verificada bajo carga real antes de habilitar cualquier actuación. La tabla resultante se serializó y el agente la cargó al arrancar.

2.5.5. Actuación y validación en modo simulado

La aplicación de niveles de frecuencia se centralizó en un componente único que registró el estado previo y garantizó su restauración incluso ante terminaciones anómalas. Antes de habilitar escrituras reales, el agente se ejecutó en modo de simulación, donde se registraron las decisiones sin modificar hardware, y se validó el comportamiento frente a corridas históricas. Solo tras estas comprobaciones se habilitó la actuación real bajo control administrativo y de reserva de recursos.

2.6. Fase 4: Validación experimental y análisis de resultados

2.6.1. Escenarios de comparación

La evaluación comparó el agente propuesto frente a tres escenarios de referencia: los gobernadores reactivos nativos (`ondemand`, `schedutil`) y un modo de frecuencia fija (`performance`). En cada escenario se ejecutó el mismo catálogo de cargas bajo las condiciones de aislamiento de la Fase 1 y se midieron tiempo de ejecución, energía, potencia media y Producto Energía–Retardo.

2.6.2. Conmutación de gobernadores y protocolo de medición

La conmutación de gobernadores se implementó verificando primero su disponibilidad en el nodo y aplicándola dentro de un contexto acotado que restauró el gobernador original al finalizar. Las condiciones de afinidad, número de hilos y asignación de núcleos se mantuvieron constantes entre escenarios para garantizar comparabilidad.

2.6.3. Cálculo del EDP y prueba de significancia

El EDP se calculó a partir de las trazas y se contrastó con las líneas base mediante las mismas pruebas pareadas empleadas en la generación de la política, reportándose los resultados desagregados por combinación de dispositivo y clase de régimen.

2.6.4. Generación del reporte comparativo

El reporte final se generó a partir de los archivos de telemetría identificados por campaña. El procedimiento manejó escenarios incompletos omitiendo comparaciones faltantes con aviso explícito, lo que permitió generar reportes parciales sin introducir supuestos.

2.7. Aspectos éticos

La presente investigación no involucra participantes humanos, datos personales ni información sensible, por lo que no requiere aprobación de un comité de ética en investigación con seres humanos. No obstante, se adoptan las siguientes consideraciones éticas, pertinentes a la naturaleza del trabajo.

En cuanto al uso responsable de infraestructura compartida, los experimentos se ejecutan sobre recursos de cómputo institucionales de acceso compartido. En consecuencia, se adopta como principio que ninguna acción del experimento debe degradar las condiciones de ejecución de terceros: las mediciones se realizan mediante asignación exclusiva otorgada por el gestor de recursos, y toda actuación sobre la frecuencia se restringe a los núcleos efectivamente asignados, verificando previamente que el dominio de frecuencia afectado no exceda dicha asignación, de acuerdo con lo expuesto en la sección 1.1.4. Del mismo modo, el estado de frecuencia del sistema se restaura al concluir la campaña, incluso ante terminaciones anómalas.

En cuanto a la integridad de los datos y de los resultados, se adopta como criterio que ninguna observación se descarte de forma silenciosa: las ventanas que no satisfacen los criterios de calidad se conservan con su anotación correspondiente, y las corridas rechazadas preservan sus artefactos. Las limitaciones conocidas del instrumento — en particular, el carácter aproximado de la estimación de bytes movidos discutido en la sección 1.1.2 — se declaran explícitamente junto con los resultados que dependen de ellas, en lugar de omitirse.

En cuanto al uso de software de terceros, las suites de benchmarks empleadas se utilizan sin modificación de su código fuente y respetando sus condiciones de distribución, y su procedencia se registra mediante la suma de verificación de los archivos originales obtenidos. Finalmente, en cuanto a transparencia y reproducibilidad, el instrumento de medición, el orquestador y las definiciones de campaña se publican en un repositorio de acceso público, junto con la documentación de las decisiones metodológicas adoptadas durante el desarrollo, de modo que los resultados puedan ser verificados o refutados por terceros.

Capítulo 3

Conclusiones

Este trabajo completó la construcción y validación empírica del instrumento de caracterización CPU–GPU de la Fase 1, pero el conjunto DVFS final aún requiere cerrar los dos recorridos experimentales pendientes. Respecto al primer objetivo específico — caracterizar cargas representativas bajo distintos estados de frecuencia —, el resultado es parcial: en CPU, la campaña limpia de reemplazo terminó con 126/126 combinaciones aceptadas e incorpora FLOPs por hardware, bytes reales de *uncore* y el ridge corregido, pero el muestreo posterior de frecuencia efectiva demostró que F0–F4 no constituyeron estados físicos distintos mientras el turbo global permaneció activo; por ello esos datos no sustentan todavía comparaciones DVFS de tiempo, energía o EDP. En GPU, el catálogo quedó caracterizado y auditado por precisión, el instrumento multi-frecuencia está implementado y `-lgc` ya retiene el reloj bajo carga real, pero falta ejecutar y aceptar la campaña completa. Los objetivos específicos restantes — entrenamiento del clasificador, implementación del agente y evaluación de EDP — corresponden a las Fases 2–4 y no se presentan como ejecutados.

El aporte metodológico central de esta fase, más allá de los datos concretos recolectados, es haber demostrado — y no solo enunciado — que ningún mecanismo de este instrumento se dio por funcional sin verificación directa contra el estado real del hardware: ni la disponibilidad de un contador de PMU, ni el presupuesto físico de contadores simultáneos, ni su persistencia en el tiempo frente a mecanismos ajenos al instrumento, ni la escritura de frecuencia de CPU, ni la precisión aritmética declarada de un kernel de GPU, ni la calibración de los propios techos del modelo Roofline. En varias ocasiones documentadas en este trabajo, esa verificación expuso una falla real que una confianza no verificada en la documentación del fabricante, en una medición previa ya dada por válida, o en el diseño original del instrumento, habría dejado pasar silenciosamente hacia el conjunto de datos.

3.1. Limitaciones

Este trabajo reconoce las siguientes limitaciones. Primera, aunque la clasificación CPU depende exclusivamente de bytes medidos mediante *uncore*, esa señal tiene un piso práctico de intervalo de aproximadamente 10 ms, más grueso que las ventanas de CPU de aproximadamente 1 ms; la intensidad y la etiqueta son, por tanto, propiedades del intervalo *uncore* difundidas a las ventanas que cubre, no mediciones independientes a 1 ms. Segunda, el presupuesto simultáneo de PMU depende del estado de mecanismos del sistema operativo ajenos al proyecto; se redujo a nueve eventos para convivir con la reserva encontrada, pero una reserva futura adicional exigiría repetir la verificación. Tercera, el control CPU por núcleo no domina el turbo global en la configuración actual: el lector por ventana permite detectar la divergencia, pero resolverla requiere autoridad administrativa sobre `no_turbo` o `max_perf_pct`; la temperatura de paquete tampoco está conectada a un gate bloqueante de sensor. Cuarta, el catálogo y los *encodings* crudos están restringidos a la microarquitectura de la Tabla 2.1; no se reclama portabilidad de los mecanismos de actuación sin repetir sus verificaciones bajo carga en cada estado de plataforma. Quinta, aunque el actuador GPU ya fue validado en dos frecuencias bajo carga, todavía no existe una campaña GPU multi-frecuencia completa aceptada, y la campaña CPU disponible no contiene niveles efectivos diferenciados.

3.2. Trabajo Futuro

En el corto plazo, tres tareas siguen directamente habilitadas. Primera, resolver con la administración el control del turbo global de CPU y repetir la campaña limpia, usando el lector por ventana para aceptar cada nivel por su frecuencia efectiva y no solo por la relectura de sysfs. Segunda, ejecutar la campaña GPU multi-frecuencia ahora que `-lgc` fue verificado bajo carga, comenzando por una prueba de humo que ejercite el mismo producto cartesiano y los mismos gates de la campaña final. Tercera, consolidar ambos conjuntos y definir la partición experimental de la Fase 2 por corrida o carga, evitando tratar como observaciones independientes ventanas que comparten una etiqueta de intervalo *uncore* o una intensidad GPU caracterizada por kernel.

Bibliografía

- [1] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 2623–2631, 2019. doi: 10.1145/3292500.3330701.
- [2] G. Ali, M. Side, S. Bhalachandra, N. J. Wright, and Y. Chen. An automated and portable method for selecting an optimal gpu frequency. *Future Generation Computer Systems*, 149:71–88, 2023. doi: 10.1016/j.future.2023.07.011.
- [3] F. Antici, A. Bartolini, Z. Kiziltan, O. Babaoglu, and Y. Kodama. Mcbound: An online framework to characterize and classify memory/compute-bound hpc jobs. In *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–15, Atlanta, GA, USA, 2024. doi: 10.1109/SC41406.2024.00062.
- [4] D. H. Bailey, E. Barszcz, J. T. Barton, D. S. Browning, R. L. Carter, L. Dagum, R. A. Fatoohi, P. O. Frederickson, T. A. Lasinski, R. S. Schreiber, H. D. Simon, V. Venkatakrisnan, and S. K. Weeratunga. The NAS parallel benchmarks. *The International Journal of Supercomputer Applications*, 5(3):63–73, 1991. doi: 10.1177/109434209100500306.
- [5] E. Barrett, C. F. Bolz-Tereick, R. Killick, S. Mount, and L. Tratt. Virtual machine warmup blows hot and cold. *Proceedings of the ACM on Programming Languages*, 1 (OOPSLA):1–27, 2017. doi: 10.1145/3133876.
- [6] David A. Beckingsale, Jason Burmark, Rich Hornung, Holger Jones, William Killian, Adam J. Kunen, Olga Pearce, Peter Robinson, Brian S. Ryujin, and Thomas R. W. Scogland. RAJA: Portable performance for large-scale scientific applications. In *2019 IEEE/ACM International Workshop on Performance, Portability and Productivity in HPC (P3HPC)*, pages 71–81, 2019. doi: 10.1109/P3HPC49587.2019.00012.
- [7] James Bergstra and Yoshua Bengio. Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13:281–305, 2012.

- [8] Leo Breiman. Random forests. *Machine Learning*, 45:5–32, 2001. doi: 10.1023/A:1010933404324.
- [9] E. Calore, A. Gabbana, S. F. Schifano, and R. Tripiccone. Evaluation of dvfs techniques on modern hpc processors and accelerators for energy-aware applications. *Concurrency and Computation: Practice and Experience*, 29(12):e4143, 2017. doi: 10.1002/cpe.4143.
- [10] Gavin C. Cawley and Nicola L. C. Talbot. On over-fitting in model selection and subsequent selection bias in performance evaluation. *Journal of Machine Learning Research*, 11:2079–2107, 2010.
- [11] Shuai Che, Michael Boyer, Jiayuan Meng, David Tarjan, Jeremy W. Sheaffer, Sang-Ha Lee, and Kevin Skadron. Rodinia: A benchmark suite for heterogeneous computing. In *2009 IEEE International Symposium on Workload Characterization (IISWC)*, pages 44–54, 2009. doi: 10.1109/IISWC.2009.5306797.
- [12] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016. doi: 10.1145/2939672.2939785.
- [13] M. T. Costa et al. Characterizing the impact of gpu power management on an exascale system. In *Proc. SC '25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC Workshops '25)*, pages 1524–1533, New York, NY, USA, 2025. doi: 10.1145/3731599.3767702.
- [14] H. David, E. Gorbato, U. R. Hanebutte, R. Khanna, and C. Le. RAPL: Memory power estimation and capping. In *Proc. 16th ACM/IEEE International Symposium on Low Power Electronics and Design (ISLPED)*, pages 189–194, 2010. doi: 10.1145/1840845.1840883.
- [15] A. Georges, D. Buytaert, and L. Eeckhout. Statistically rigorous java performance evaluation. In *Proc. 22nd Annual ACM SIGPLAN Conference on Object-Oriented Programming Systems and Applications (OOPSLA '07)*, pages 57–76, Montreal, QC, Canada, 2007. doi: 10.1145/1297027.1297033.
- [16] Pierre Geurts, Damien Ernst, and Louis Wehenkel. Extremely randomized trees. *Machine Learning*, 63(1):3–42, 2006. doi: 10.1007/s10994-006-6226-1.
- [17] R. Gonzalez, B. M. Gordon, and M. A. Horowitz. Supply and threshold voltage scaling for low power CMOS. *IEEE Journal of Solid-State Circuits*, 32(8):1210–1216, 1997. doi: 10.1109/4.604018.

- [18] Kazushige Goto and Robert A. van de Geijn. Anatomy of high-performance matrix multiplication. *ACM Transactions on Mathematical Software*, 34(3):1–25, 2008. doi: 10.1145/1356052.1356053. Article 12.
- [19] B. Gregg. *Systems Performance: Enterprise and the Cloud*. Prentice Hall, Upper Saddle River, NJ, USA, 2nd edition, 2020.
- [20] J. Guerreiro, A. Ilic, N. Roma, and P. Tomás. DVFS-aware application classification to improve GPGPUs energy efficiency. *Parallel Computing*, 83:93–117, 2019. doi: 10.1016/j.parco.2018.02.001.
- [21] R. Hebbar and A. Milenković. PMU-events-driven DVFS techniques for improving energy efficiency of modern processors. *ACM Transactions on Modeling and Performance Evaluation of Computing Systems*, 7(1):1–31, 2022. doi: 10.1145/3538645.
- [22] Intel Corporation. *3rd Gen Intel Xeon Processor Scalable Family, Codename Ice Lake, Uncore Performance Monitoring Reference Manual*, May 2021. URL <https://cdrdv2-public.intel.com/679093/639778%20ICX%20UPG%20v1.pdf>. Revision 1.0.
- [23] Intel Corporation. perfmon: Performance monitoring events for intel architecture processors (evento fp_arith_inst_retired, familia ice lake), 2026. URL <https://github.com/intel/perfmon>. Accessed: Aug. 10, 2026.
- [24] Intel Corporation. Intel 64 and ia-32 architectures software developer’s manual, volume 3b: System programming guide, part 2 (performance monitoring, nmi watchdog interaction with fixed-function and general-purpose counters), 2026. URL <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>. Accessed: Aug. 12, 2026.
- [25] Gareth James, Daniela Witten, Trevor Hastie, and Robert Tibshirani. *An Introduction to Statistical Learning: with Applications in R*. Springer Texts in Statistics. Springer, 2nd edition, 2021. doi: 10.1007/978-1-0716-1418-1.
- [26] Shachar Kaufman, Saharon Rosset, Claudia Perlich, and Ori Stitelman. Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data*, 6(4):1–21, 2012. doi: 10.1145/2382577.2382579.
- [27] M. Kerrisk. *The Linux Programming Interface: A Linux and UNIX System Programming Handbook*. No Starch Press, San Francisco, CA, USA, 2010.

- [28] J. H. Laros III, K. Pedretti, S. M. Kelly, W. Shu, K. Ferreira, J. Van Dyke, and C. Vaughan. Energy delay product. In *Energy-Efficient High Performance Computing: Measurement and Tuning*, SpringerBriefs in Computer Science, chapter 6, pages 51–55. Springer, London, UK, 2013. doi: 10.1007/978-1-4471-4492-2_9.
- [29] Linux Kernel Documentation. CPU Performance Scaling, 2026. URL <https://docs.kernel.org/admin-guide/pm/cpufreq.html>. Accessed: Sep. 5, 2026.
- [30] Linux Kernel Documentation. `intel_pstate` cpu performance scaling driver, 2026. URL https://docs.kernel.org/admin-guide/pm/intel_pstate.html. Accessed: Sep. 5, 2026.
- [31] Linux man-pages project. `perf_event_open(2)` — linux manual page, 2026. URL https://man7.org/linux/man-pages/man2/perf_event_open.2.html. Accessed: Aug. 29, 2026.
- [32] T.-Y. Liu, J. Guo, and B. Huang. Efficient cross-platform multiplexing of hardware performance counters via adaptive grouping. *ACM Transactions on Architecture and Code Optimization*, 21(1):1–26, 2024. doi: 10.1145/3629525.
- [33] Yu Jung Lo, Samuel Williams, Brian Van Straalen, Terry J. Ligocki, Matthew J. Cordery, Nicholas J. Wright, Mary W. Hall, and Leonid Oliker. Roofline model toolkit: A practical tool for architectural and program analysis. In *High Performance Computing Systems. Performance Modeling, Benchmarking, and Simulation (PMBS 2014)*, volume 8966 of *Lecture Notes in Computer Science*, pages 129–148. Springer, 2015. doi: 10.1007/978-3-319-17248-4_7.
- [34] John D. McCalpin. Memory bandwidth and machine balance in current high performance computers. Technical report, IEEE Computer Society Technical Committee on Computer Architecture (TCCA) Newsletter, December 1995.
- [35] S. McCarty. Architecting containers part 1: Why understanding user space vs. kernel space matters. Red Hat Blog, 2015. URL <https://www.redhat.com/en/blog/architecting-containers-part-1-why-understanding-user-space-vs-kernel-space-matters>. Accessed: Apr. 3, 2025.
- [36] NVIDIA. Nvidia management library (nvml). NVIDIA Developer Documentation, 2026. URL <https://docs.nvidia.com/deploy/nvml-api/index.html>. Accessed: Apr. 3, 2026.

- [37] NVIDIA. Nsight compute. NVIDIA Developer Documentation, 2026. URL <https://docs.nvidia.com/nsight-compute/>. Accessed: Aug. 8, 2026.
- [38] David R. Roberts, Volker Bahn, Simone Ciuti, Mark S. Boyce, Jane Elith, Gurutzeta Guillera-Arroita, Severin Hauenstein, José J. Lahoz-Monfort, Boris Schröder, Wilfried Thuiller, David I. Warton, Brendan A. Wintle, Florian Hartig, and Carsten F. Dormann. Cross-validation strategies for data with temporal, spatial, hierarchical, or phylogenetic structure. *Ecography*, 40(8):913–929, 2017. doi: 10.1111/ecog.02881.
- [39] O. S. Simsek, J.-G. Piccinali, and F. M. Ciorba. Increasing energy efficiency of astrophysics simulations through gpu frequency scaling. In *SC24-W: Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1826–1834, Atlanta, GA, USA, 2024. doi: 10.1109/SCW63240.2024.00229.
- [40] Marina Sokolova and Guy Lapalme. A systematic analysis of performance measures for classification tasks. *Information Processing & Management*, 45(4):427–437, 2009. doi: 10.1016/j.ipm.2009.03.002.
- [41] P. Thamm and U. Leser. Strategies to measure energy consumption using RAPL during workflow execution on commodity clusters. *arXiv preprint arXiv:2505.09375*, 2025.
- [42] J. Treibig, G. Hager, and G. Wellein. LIKWID: A lightweight performance-oriented tool suite for x86 multicore environments. In *Proc. 39th International Conference on Parallel Processing Workshops (ICPPW)*, pages 207–216, San Diego, CA, USA, 2010. doi: 10.1109/ICPPW.2010.38.
- [43] D. Velicka, O. Vysocky, and L. Riha. Methodology for GPU frequency switching latency measurement. In *Proc. 2025 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, pages 830–839, 2025. doi: 10.1109/IPDPSW66978.2025.00133.
- [44] Y. Wang, M. Hao, H. He, W. Zhang, Q. Tang, X. Sun, and Z. Wang. DRLCap: Runtime GPU frequency capping with deep reinforcement learning. *IEEE Transactions on Sustainable Computing*, 9(5):712–726, 2024. doi: 10.1109/TSUSC.2024.3361596.
- [45] V. M. Weaver. Self-monitoring overhead of the linux perf event performance counter interface. In *Proc. IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 102–111, 2015.

- [46] S. Williams, A. Waterman, and D. Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009. doi: 10.1145/1498765.1498785.

Repositorio y recursos complementarios

Repositorio público del proyecto:

<https://github.com/AdrianCCRS/hyperion>